Computer Organisation & Program Execution 2019

5

Asynchronism

Uwe R. Zimmer - The Australian National University

References for this chapter

[Patterson17]

David A. Patterson & John L. Hennessy
Computer Organization and Design – The Hardware/Software Interface
Chapter 4 "The Processor",
Chapter 6 "Parallel Processors from Client to Cloud"
ARM edition, Morgan Kaufmann 2017

Why?

How do you handle your communication flow?

Why?

How do you handle your communication flow?

☞ Do you have times when you check certain communication?

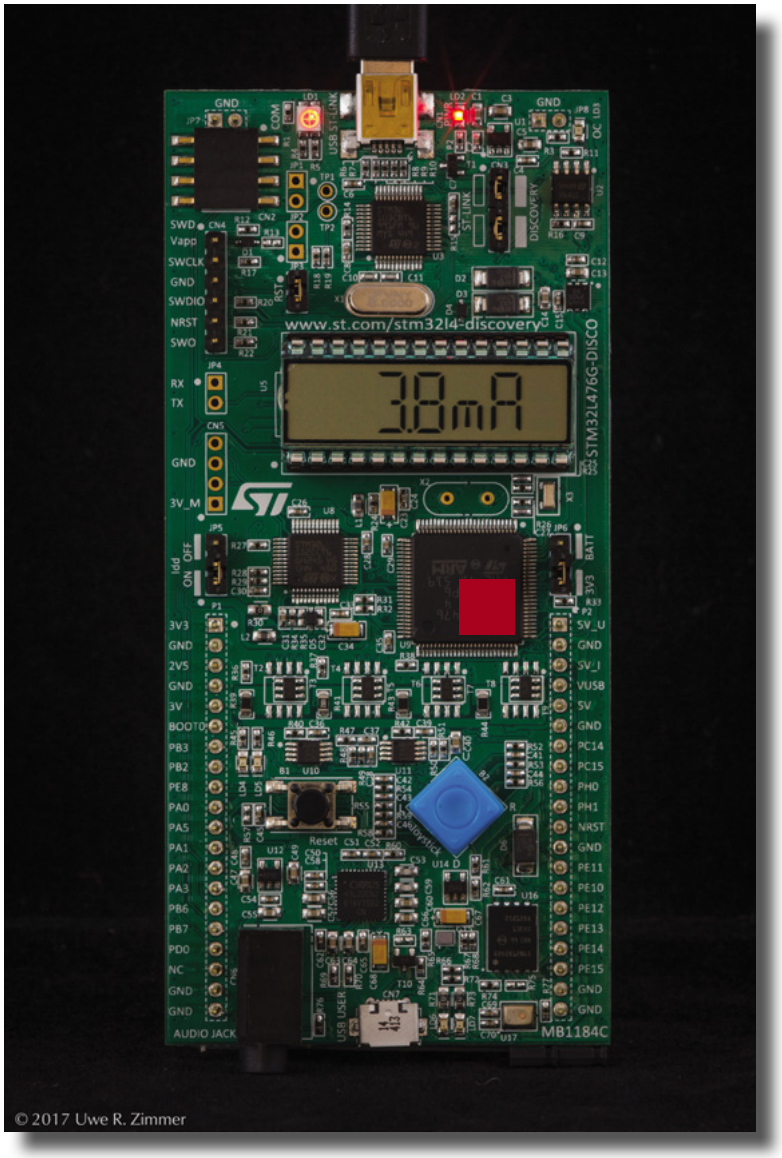
☞ Is certain communication interrupting you? – at any time?

☞ Do you assign "importance levels" to your communication channels/sources?

STM32L476 Discovery

CPU

… running its sequence of machine instructions.



STM32L476 Discovery

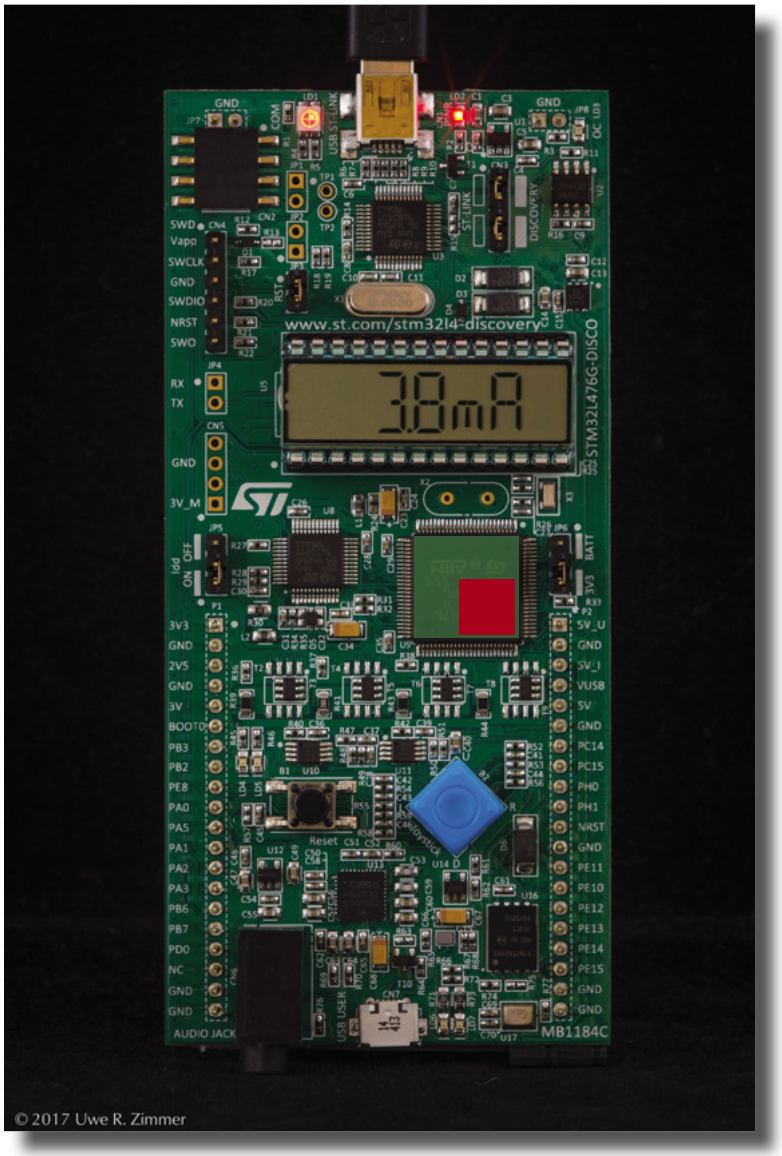
CPU

… running its sequence of machine instructions.

☞ How to interact with all the other devices inside the

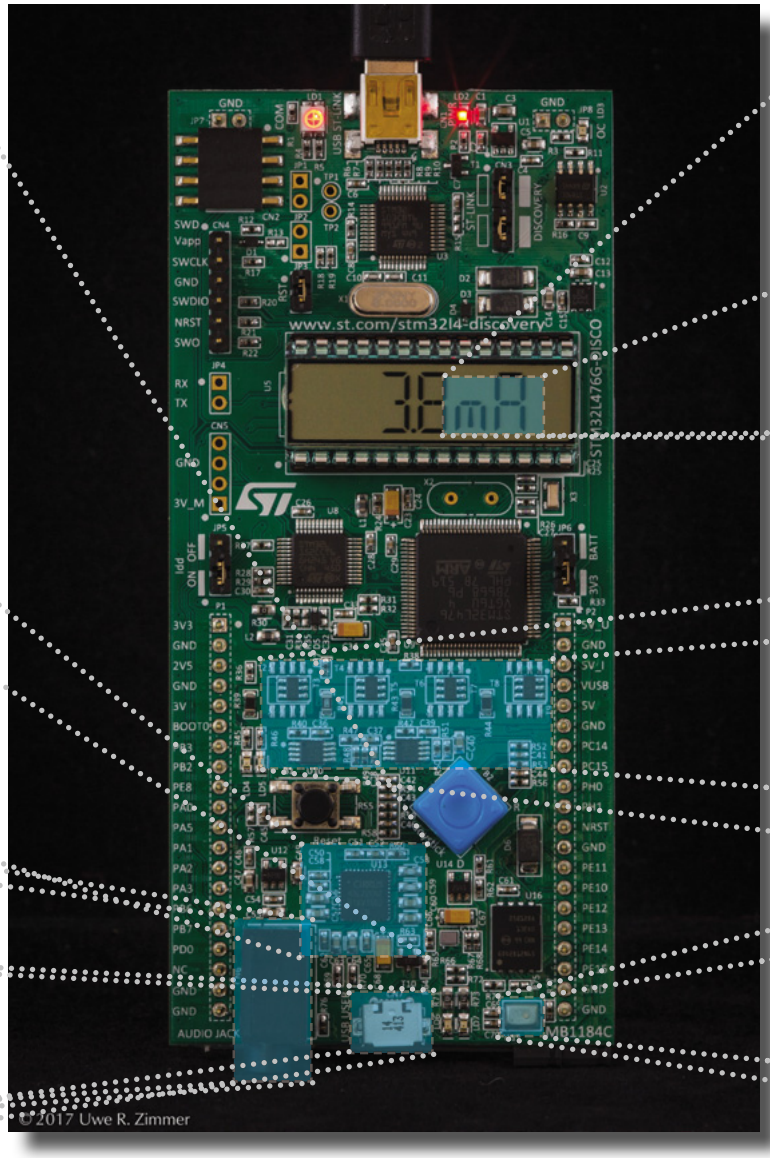
MCU

?



STM32L476 Discovery

Multiplexed 24 bit
ΣΔ-DAConverter with
stereo power amp



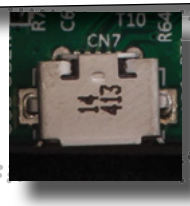
"9 axis" motion sensor
(underneath display):

- 3 axis accelerometer
- 3 axis gyroscope
- 3 axis magnetometer

Current meter to MCU
60 nA … 50 mA

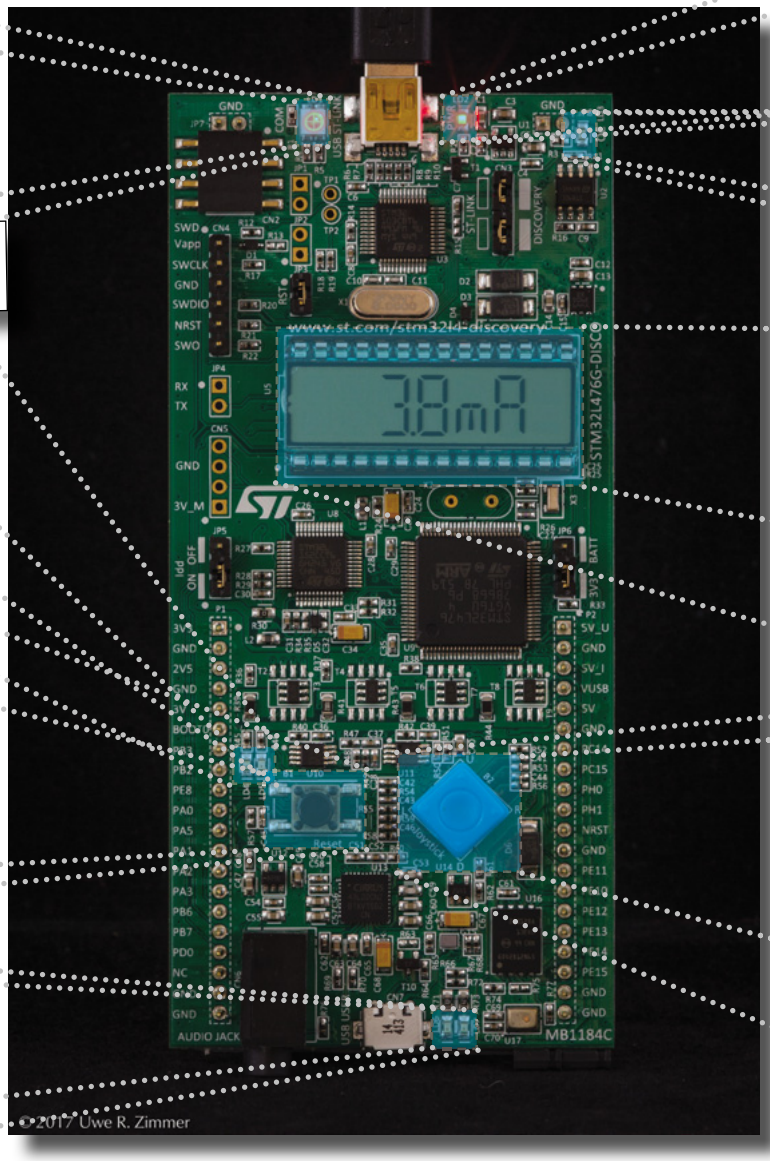
Headphone jack

USB OTG

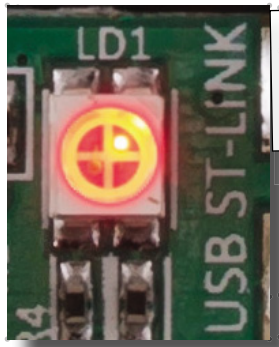


Microphone

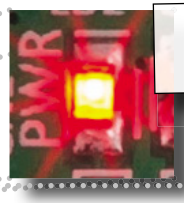
STM32L476 Discovery



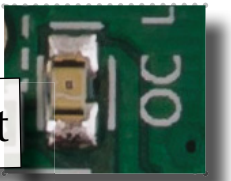
Debugger state



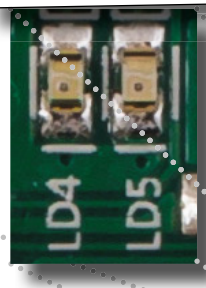
Power



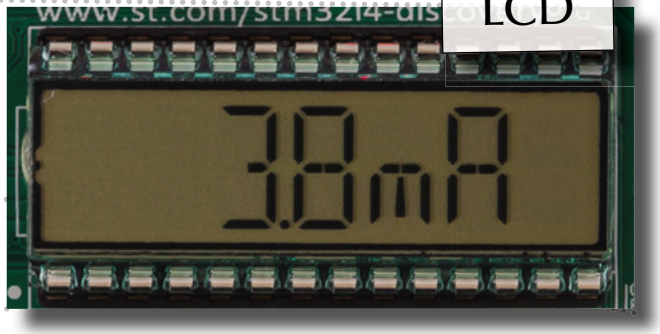
Over current



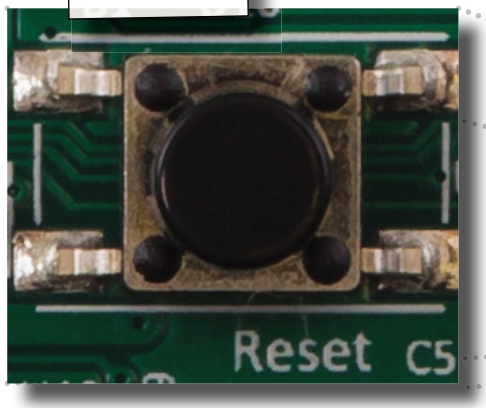
User LEDs



LCD

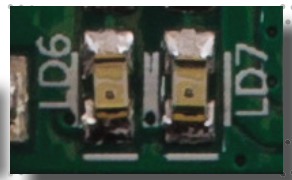


Reset



Joystick

OTG LEDs



STM32L476 Discovery

CPU

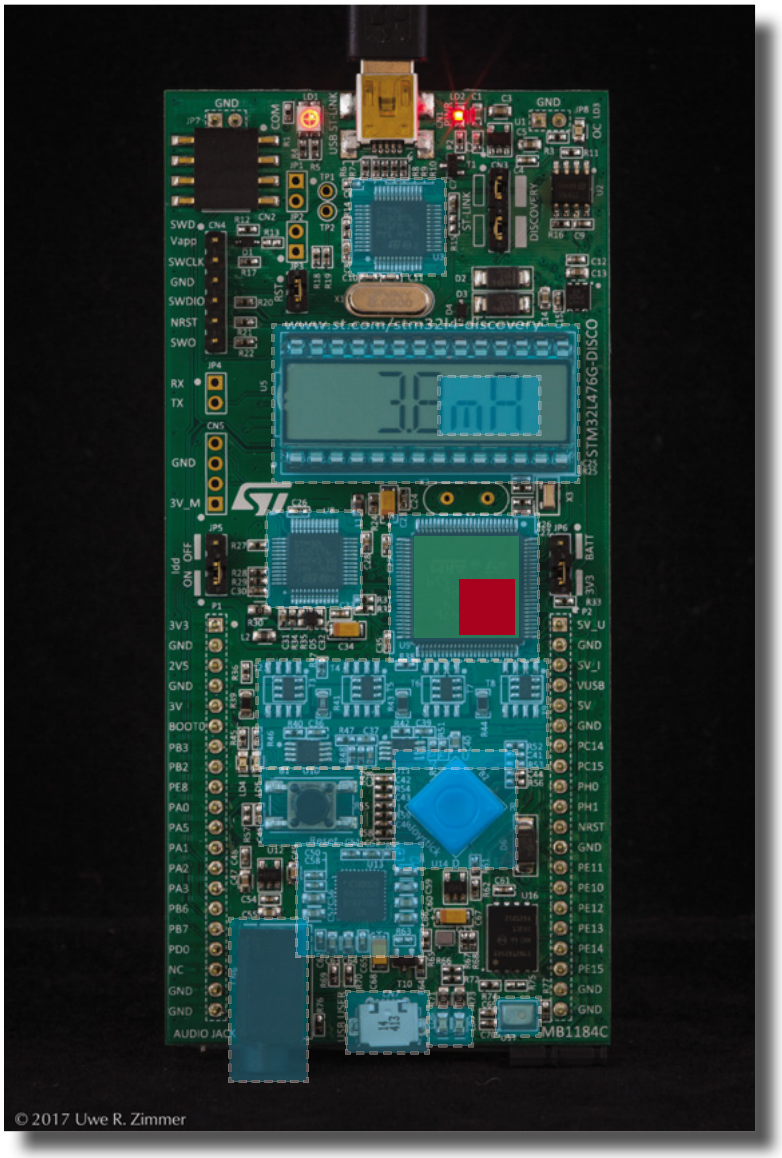
… running its sequence of machine instructions.

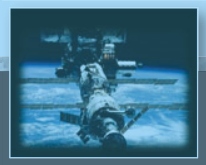
☞ How to interact with all the other devices inside the

MCU

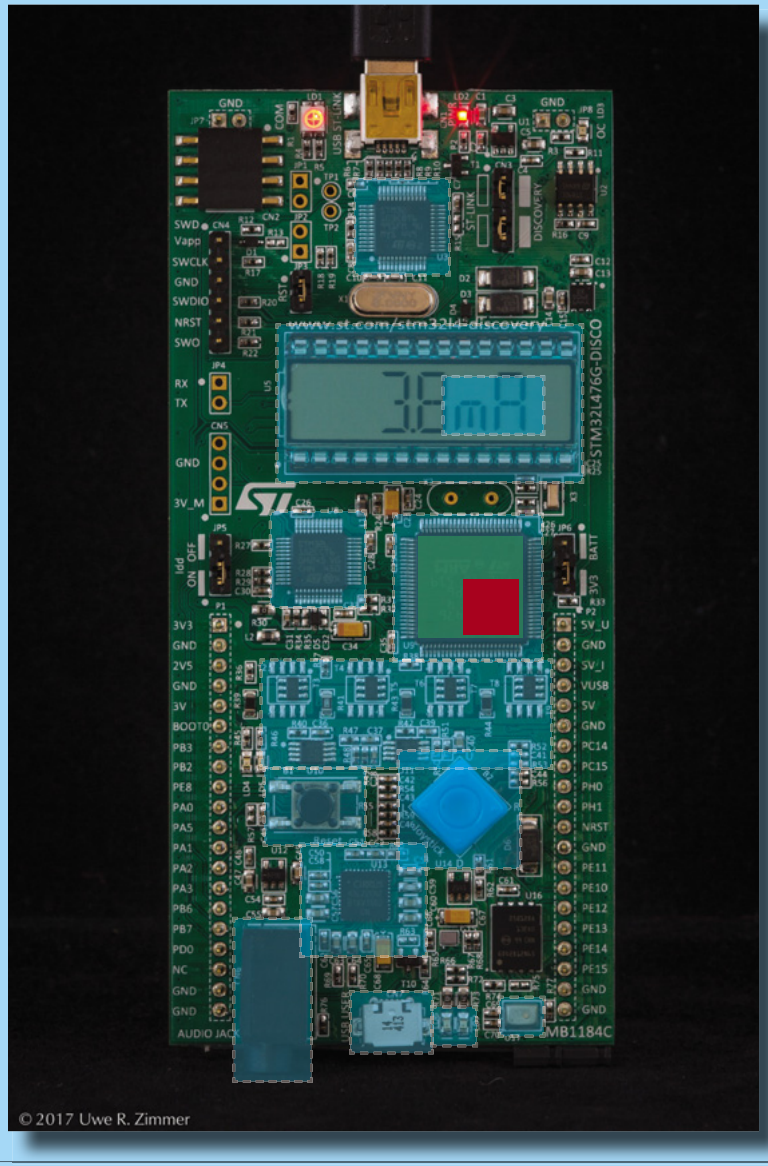
?

☞ … and then with all the devices on the board?





STM32L476 Discovery



CPU

… running its sequence of machine instructions.

☞ How to interact with all the other devices inside the

MCU

?

☞ … and then with all the devices on the board?

☞ and then with the

rest of the world

… which is connected to the board?

Polling

Sequential machine instructions

- ☞ All external devices need to be “checked” by asking for their status.
- ☞ This should usually happen (semi-) regularly.

Polling

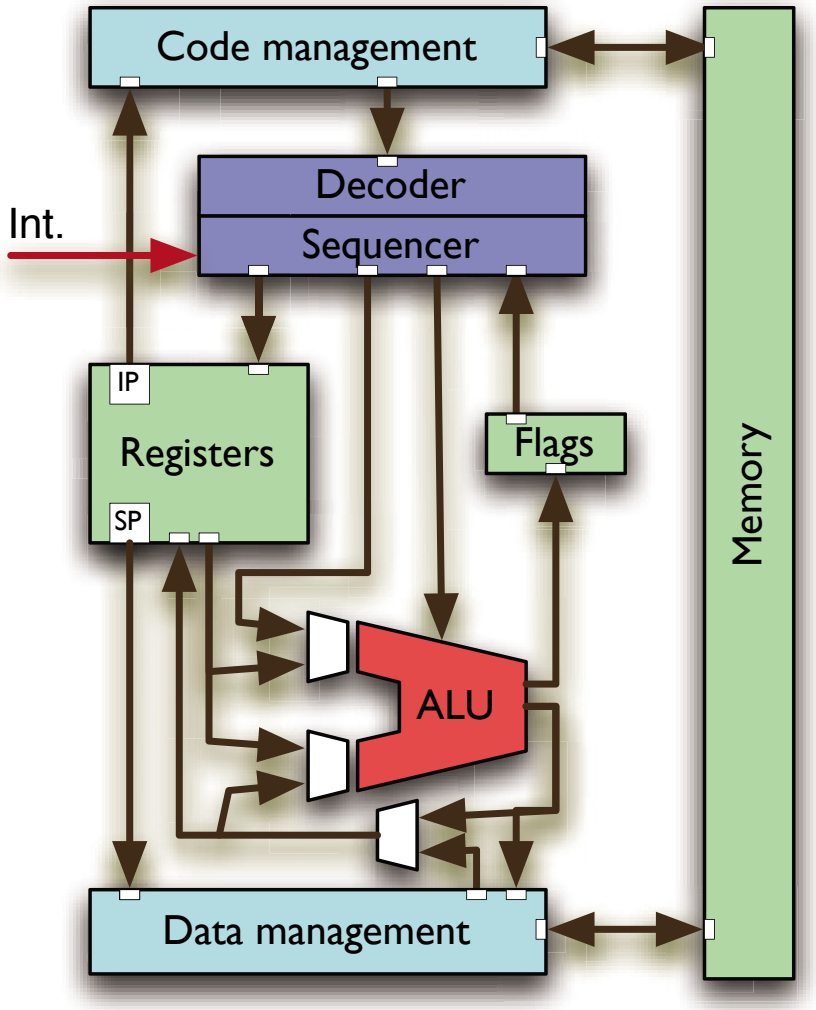
Sequential machine instructions

- ☞ All external devices need to be “checked” by asking for their status.
- ☞ This should usually happen (semi-) regularly.

☞ This will lead to a loop of **polling** requests.

- ↗ Maximal latencies can be calculated straight forward.
- ↗ Simplicity of design (with small number of devices).
- ↗ Fastest option with small number of devices (like: one).
- ↘ All devices will need to wait their turn
  … even if this device is the only one with new data!
- ↘ The “main” program transforms into one large loop which can be hard to handle in terms of scalable program design.
- ↘ Events or data can be missed.

Processor Architectures



Interrupts

- One or multiple lines wired directly into the sequencer

☞ *Required for*:
Pre-emptive scheduling, **Timer driven actions**, **Transient hardware interactions**, …

☞ Usually preceded by an external logic ("interrupt controller") which accumulates and encodes all external requests.

On interrupt (if unmasked):

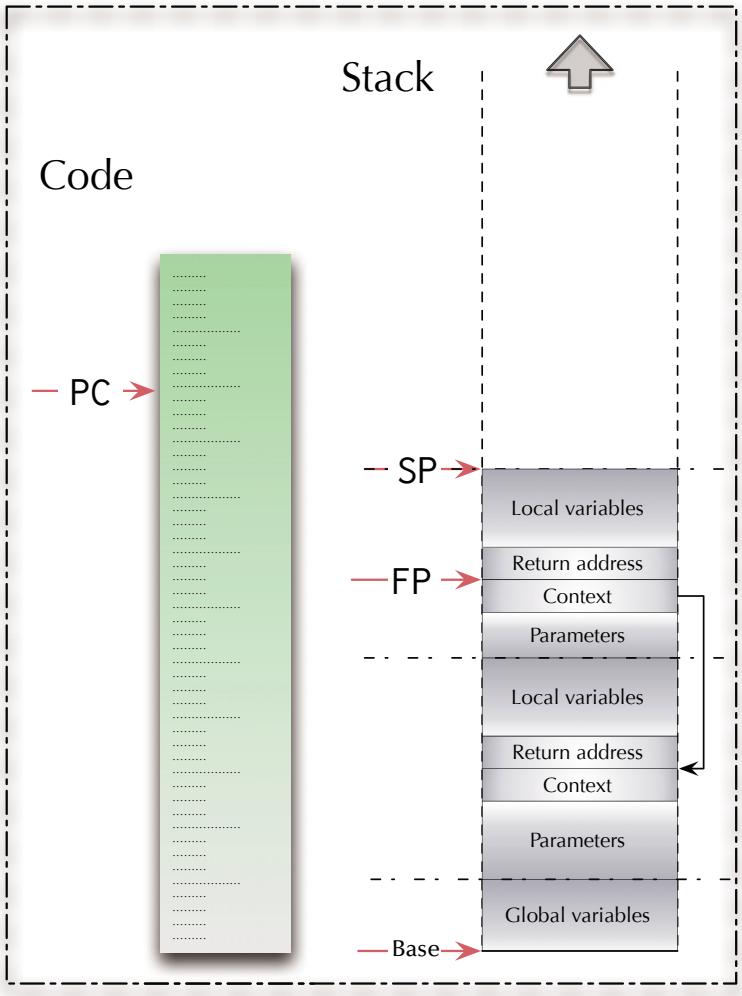
- CPU stops normal sequencer flow.
- Lookup of interrupt handler's address
- Current IP and state pushed onto stack.
- IP set to interrupt handler.

Bahia Honda Rail Bridge (Creative Commons Attribution-ShareAlike 3.0, Photography by MrX at English Wikipedia)

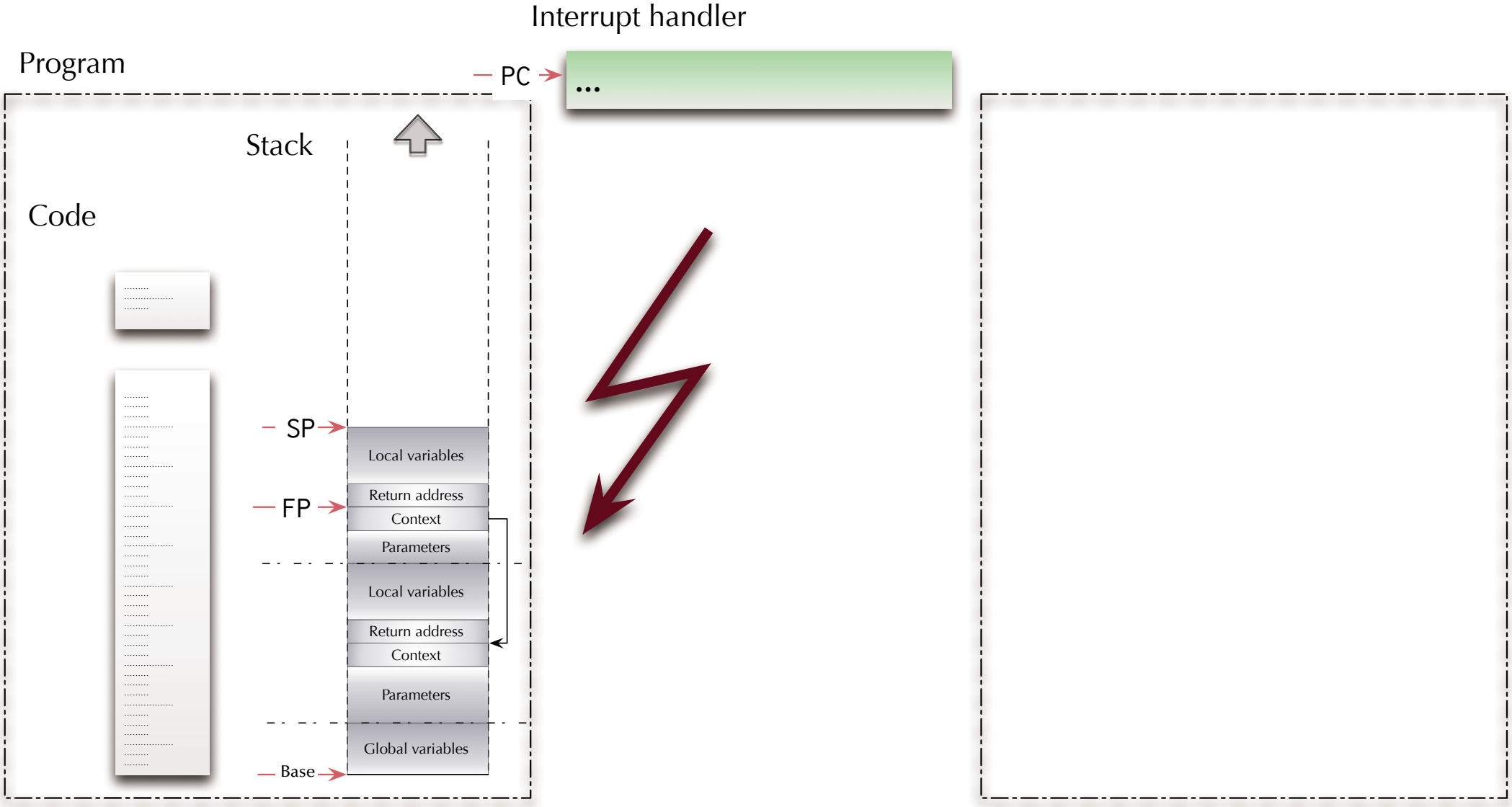
Interrupt processing

Interrupt handler

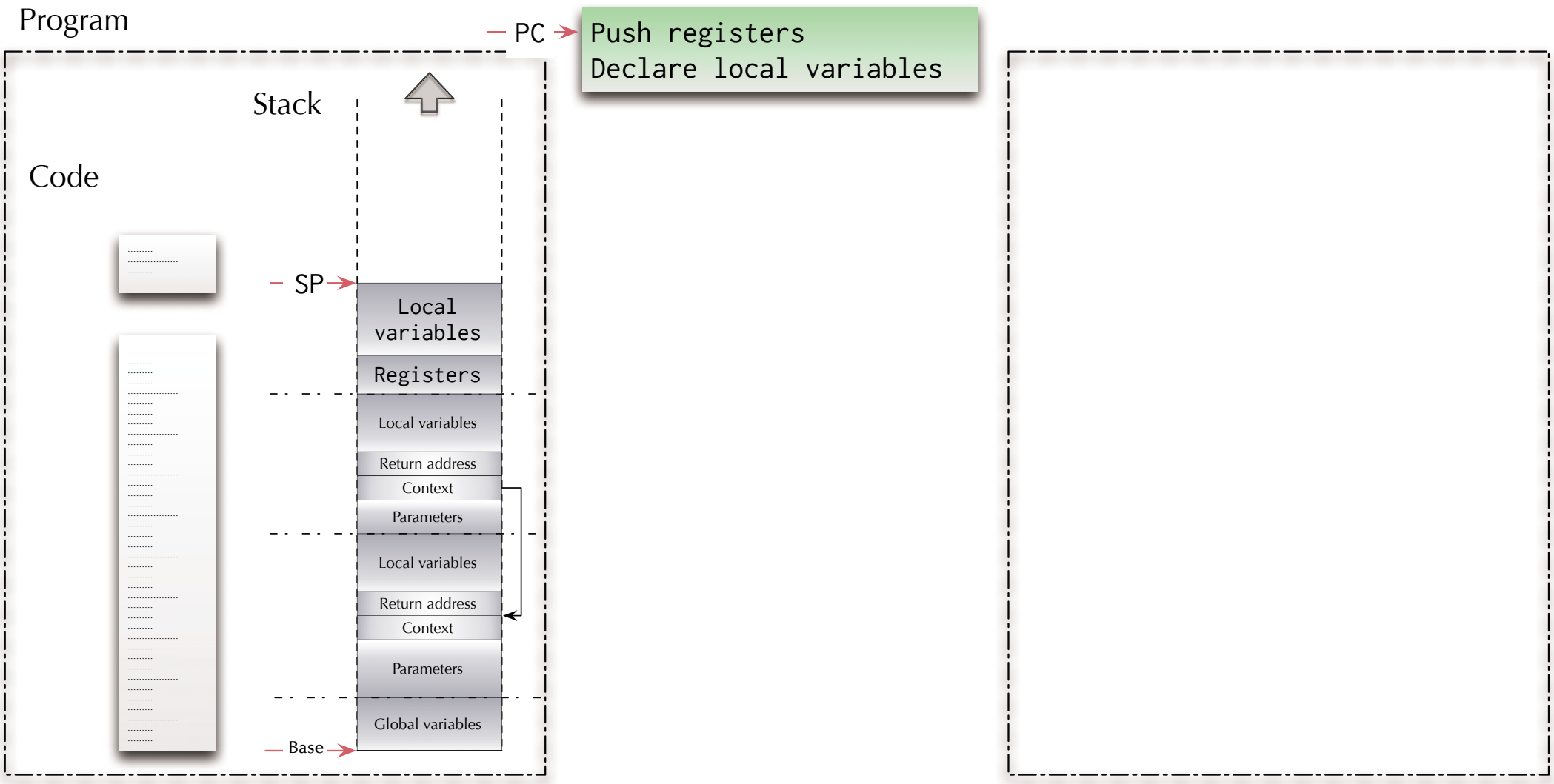
Program



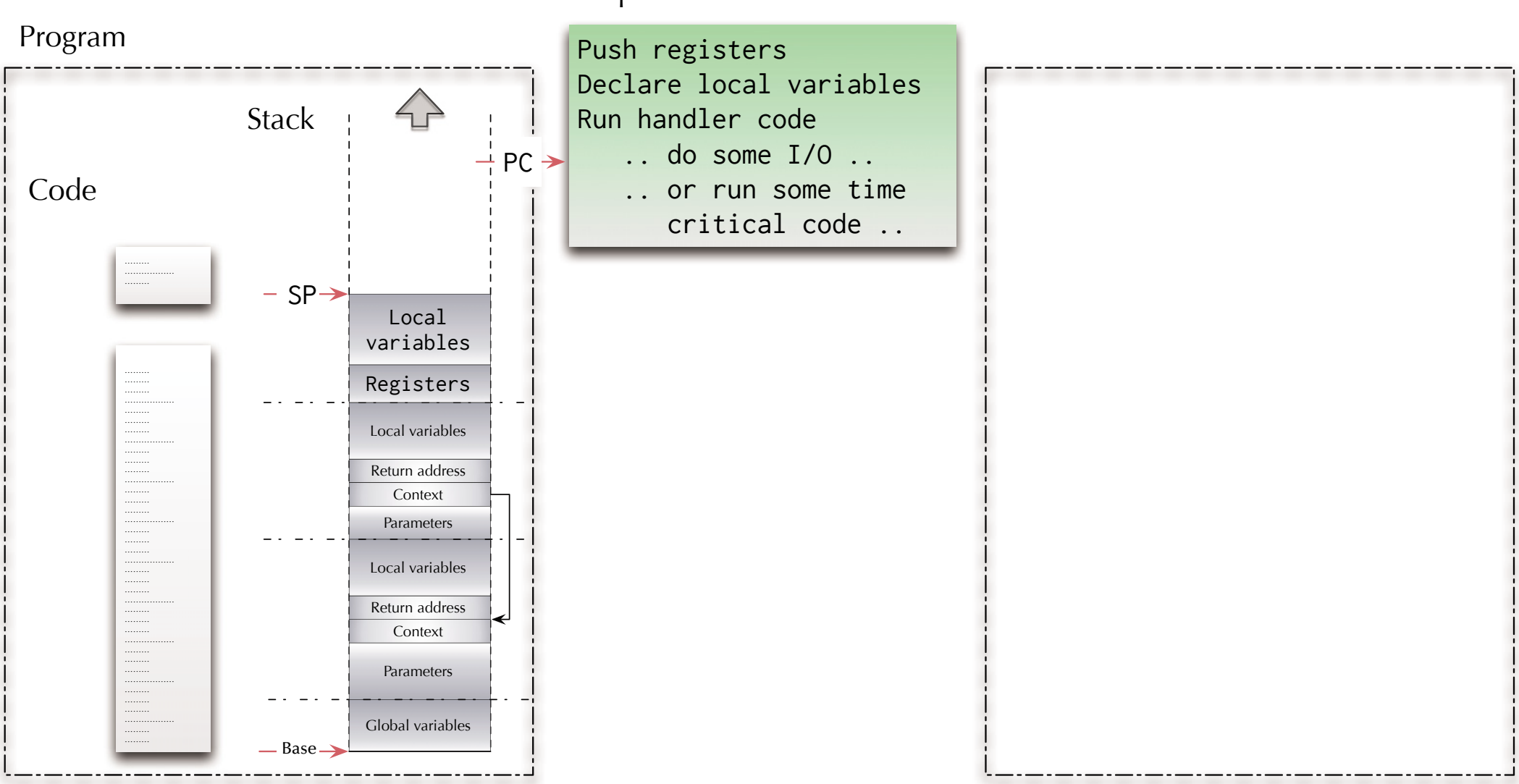
Interrupt processing



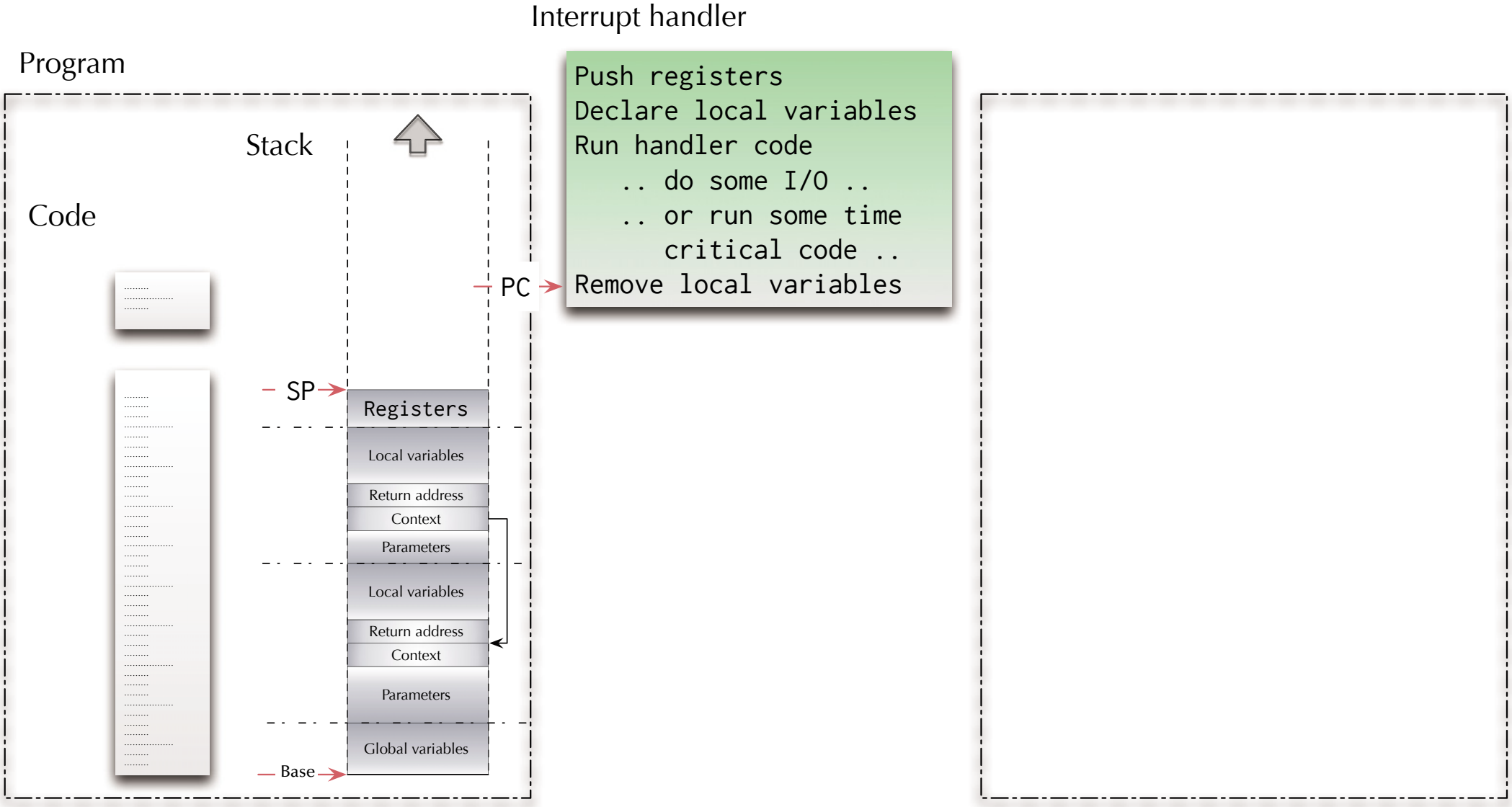
Interrupt processing



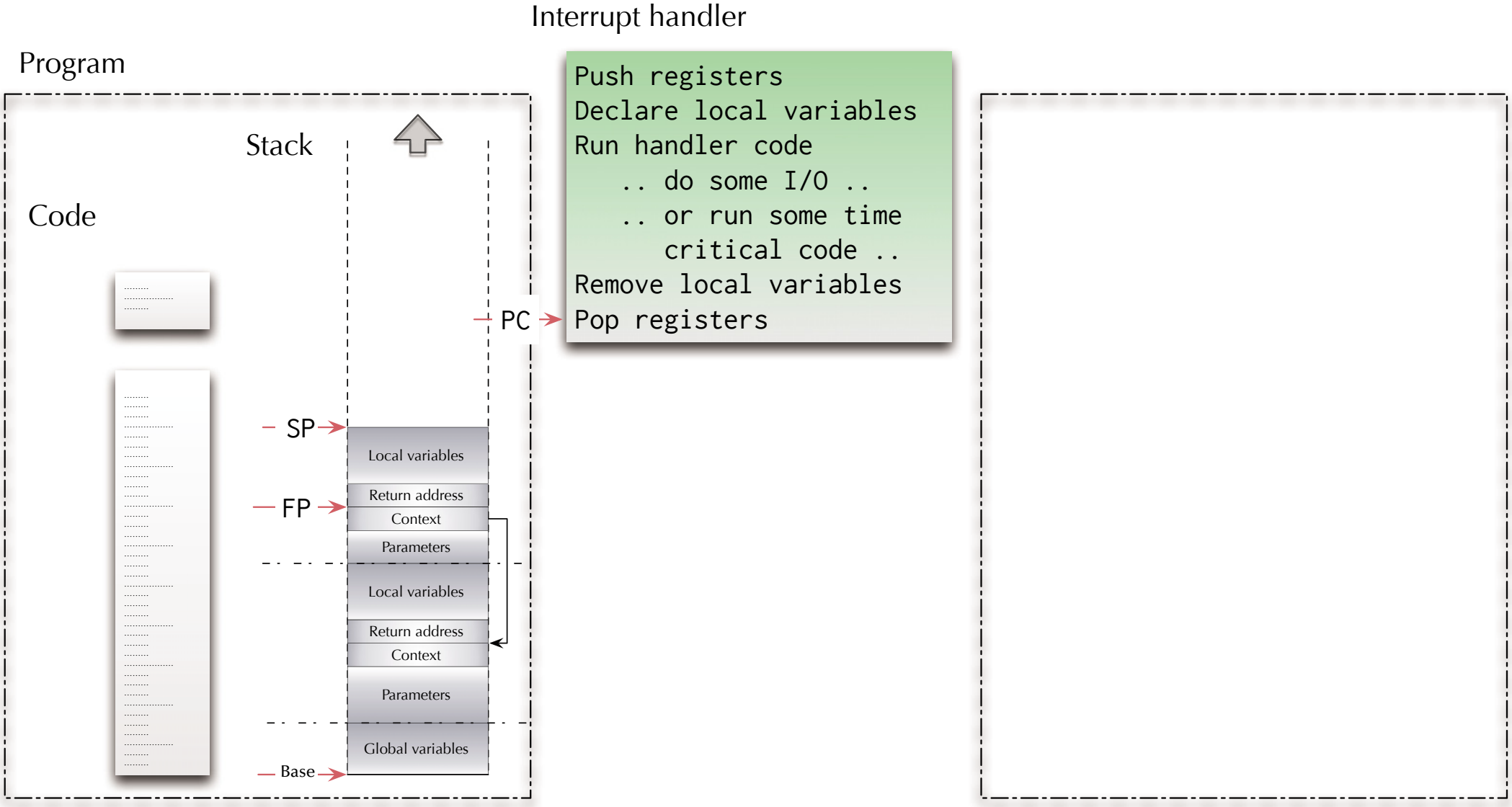
Interrupt processing



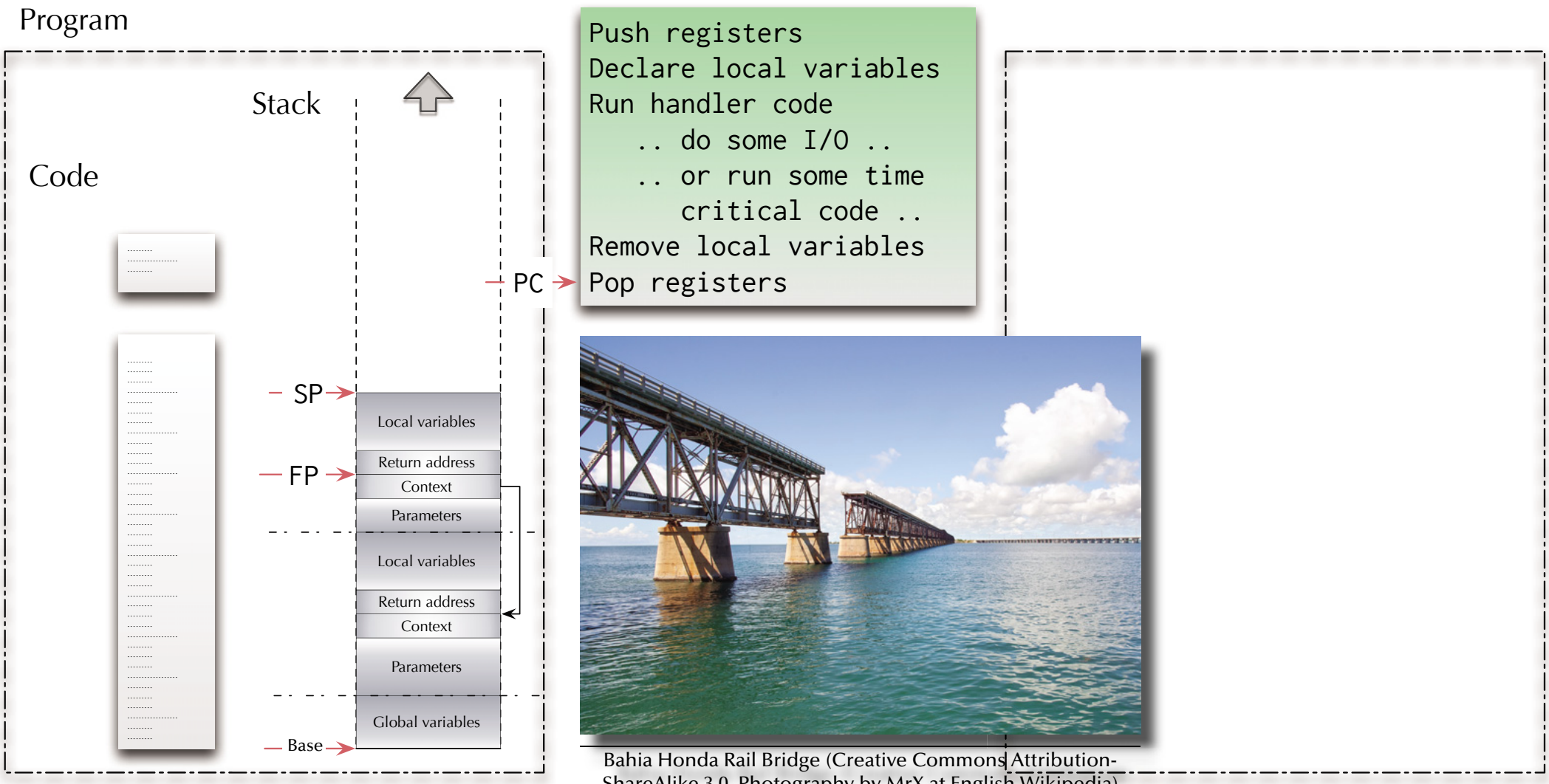
Interrupt processing



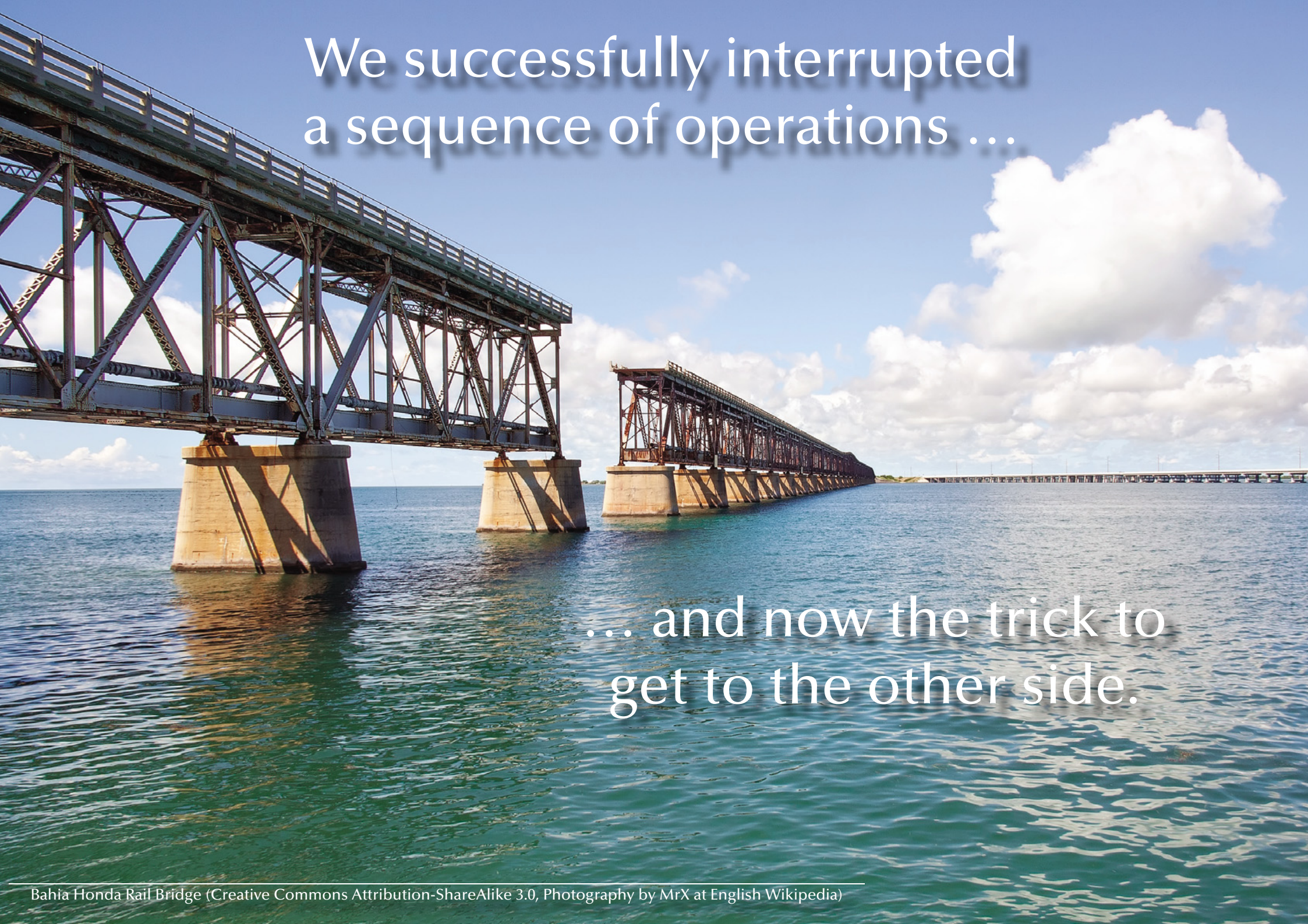
Interrupt processing



Interrupt processing



Bahia Honda Rail Bridge (Creative Commons Attribution-ShareAlike 3.0, Photography by MrX at English Wikipedia)

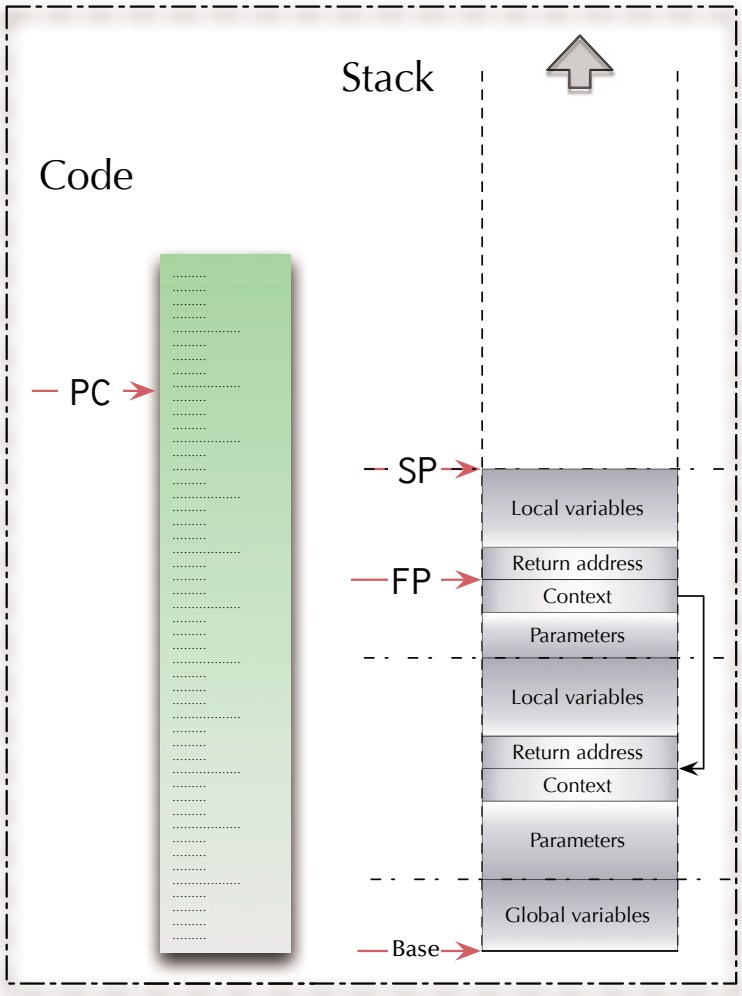


Bahia Honda Rail Bridge (Creative Commons Attribution-ShareAlike 3.0, Photography by MrX at English Wikipedia)

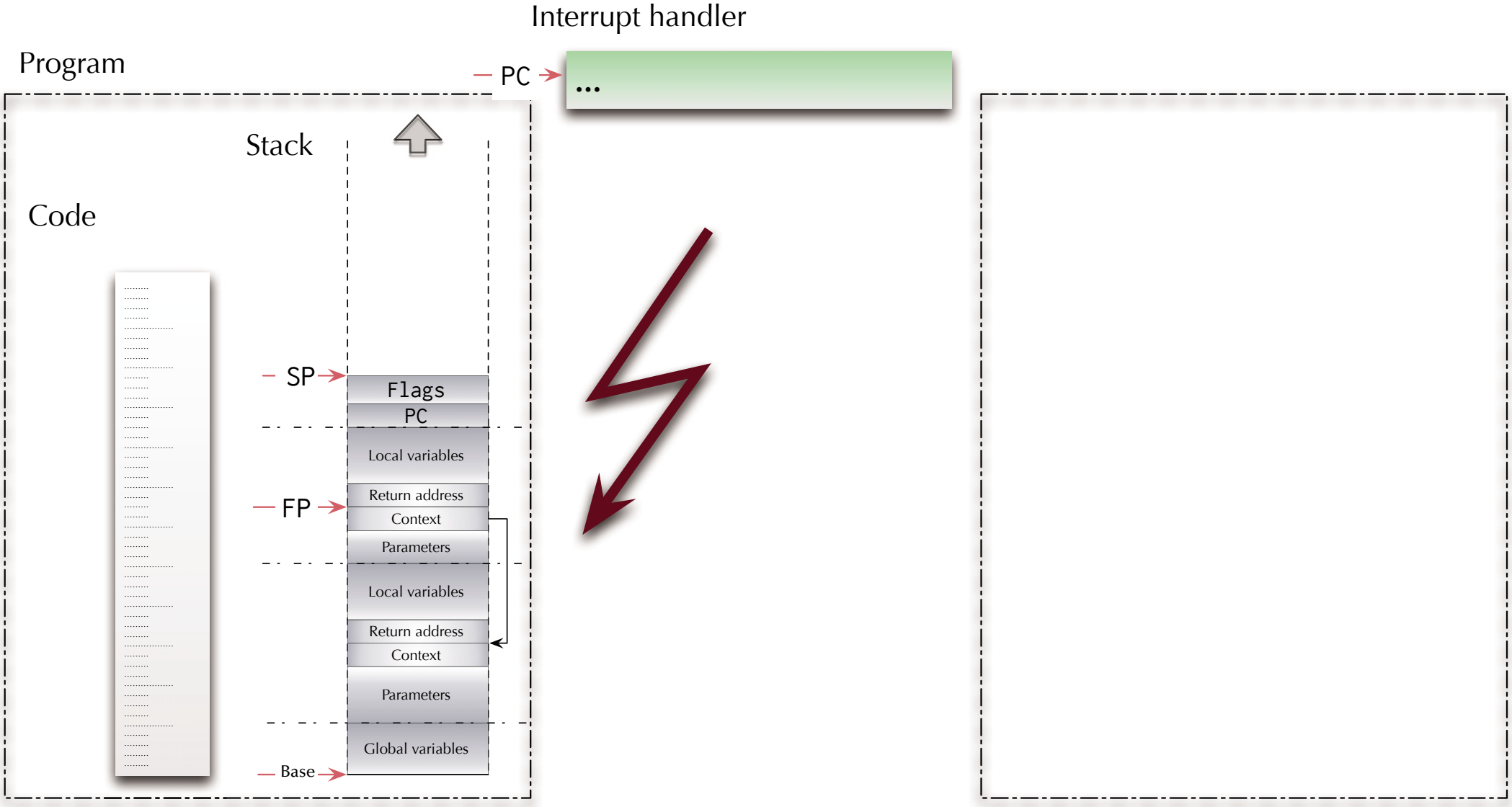
Interrupt processing

Interrupt handler

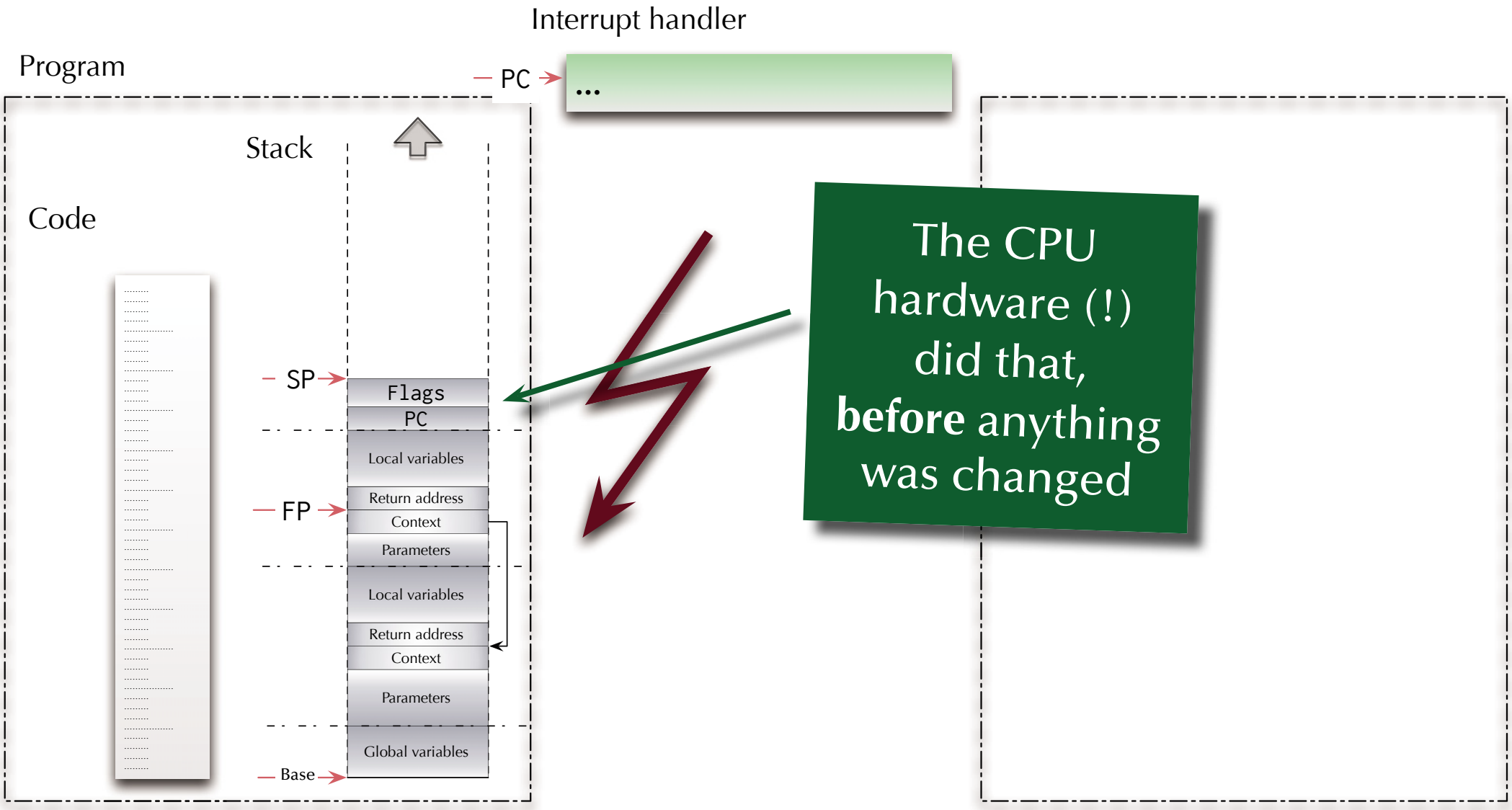
Program



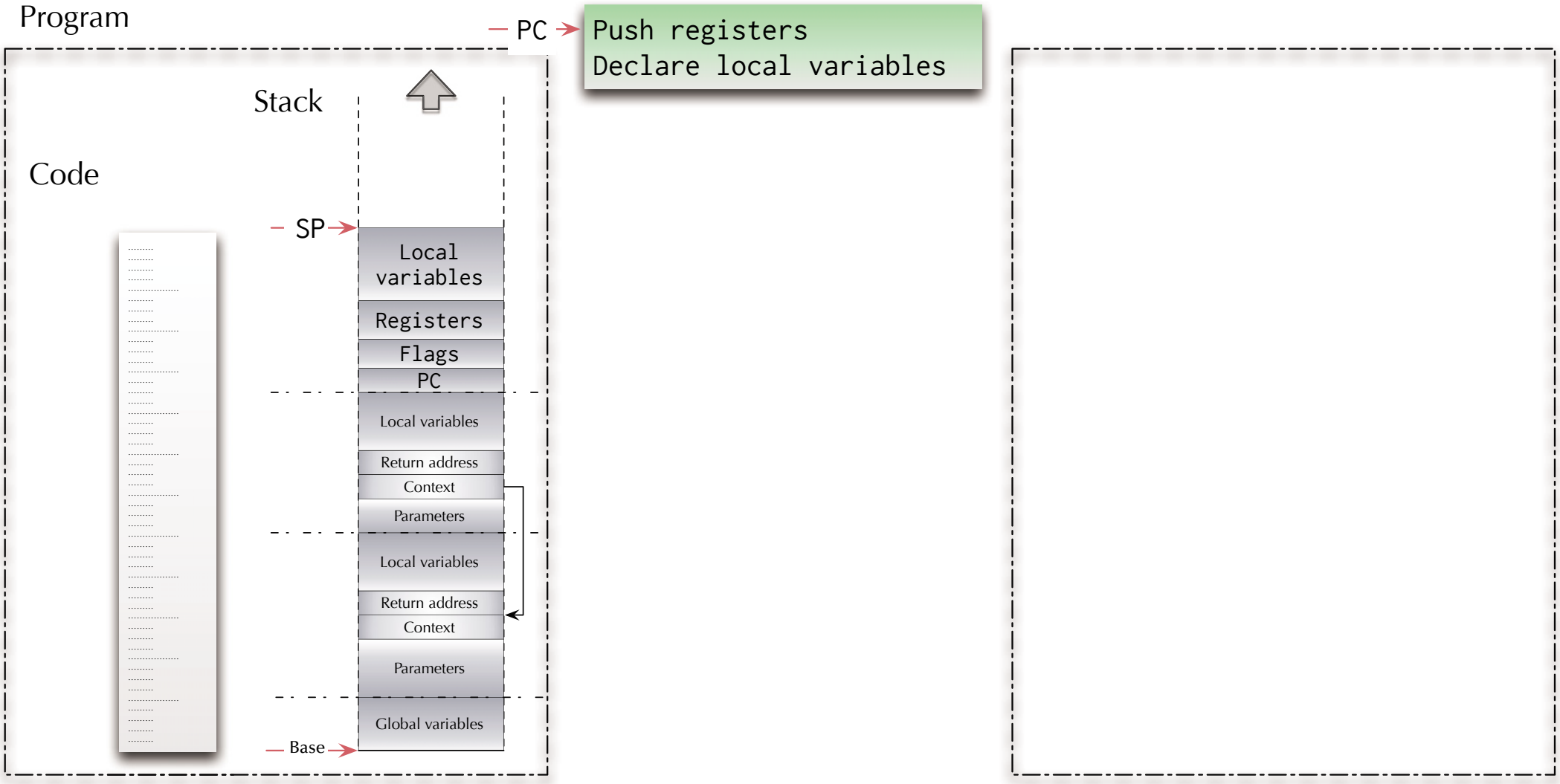
Interrupt processing



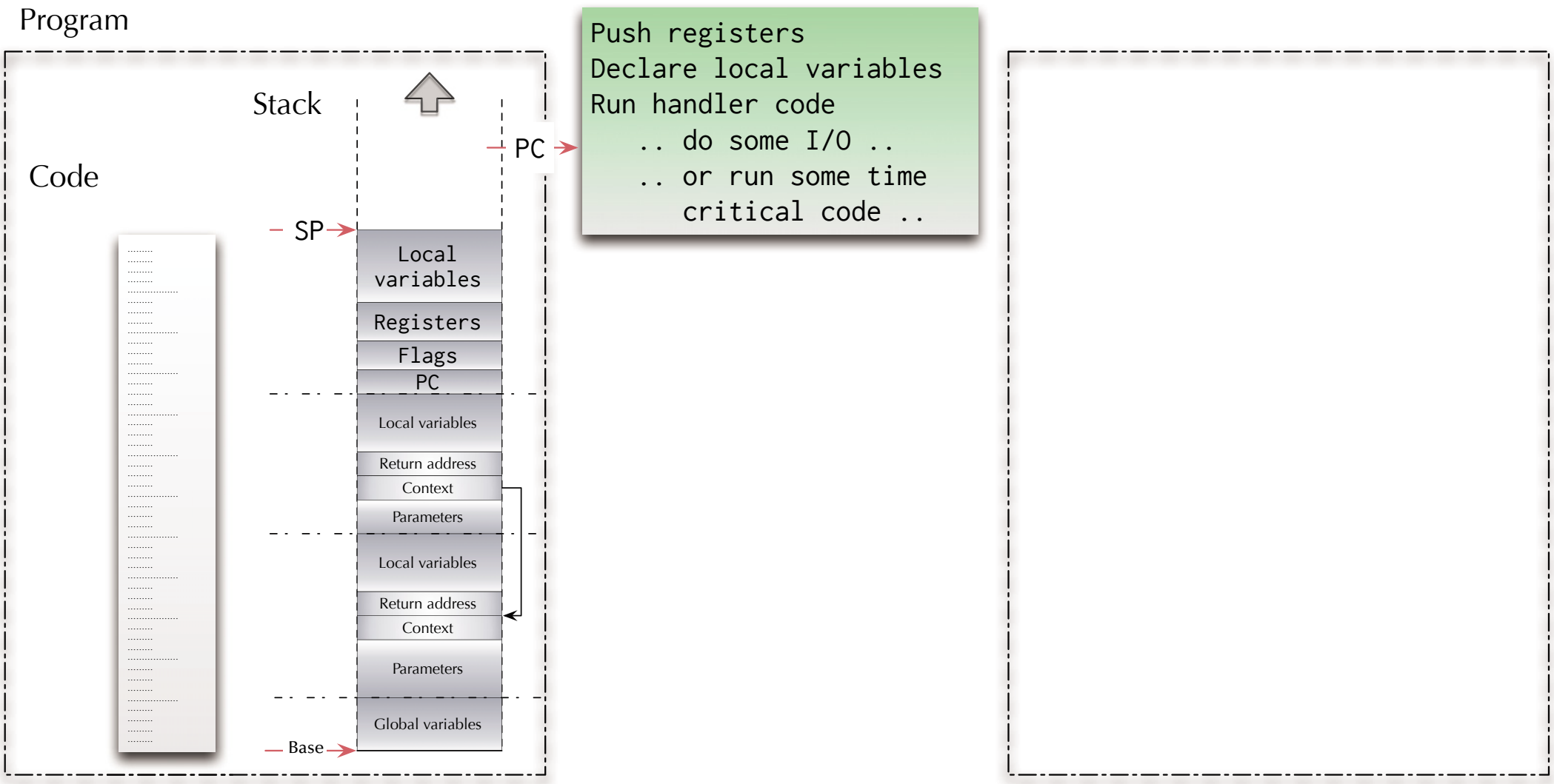
Interrupt processing



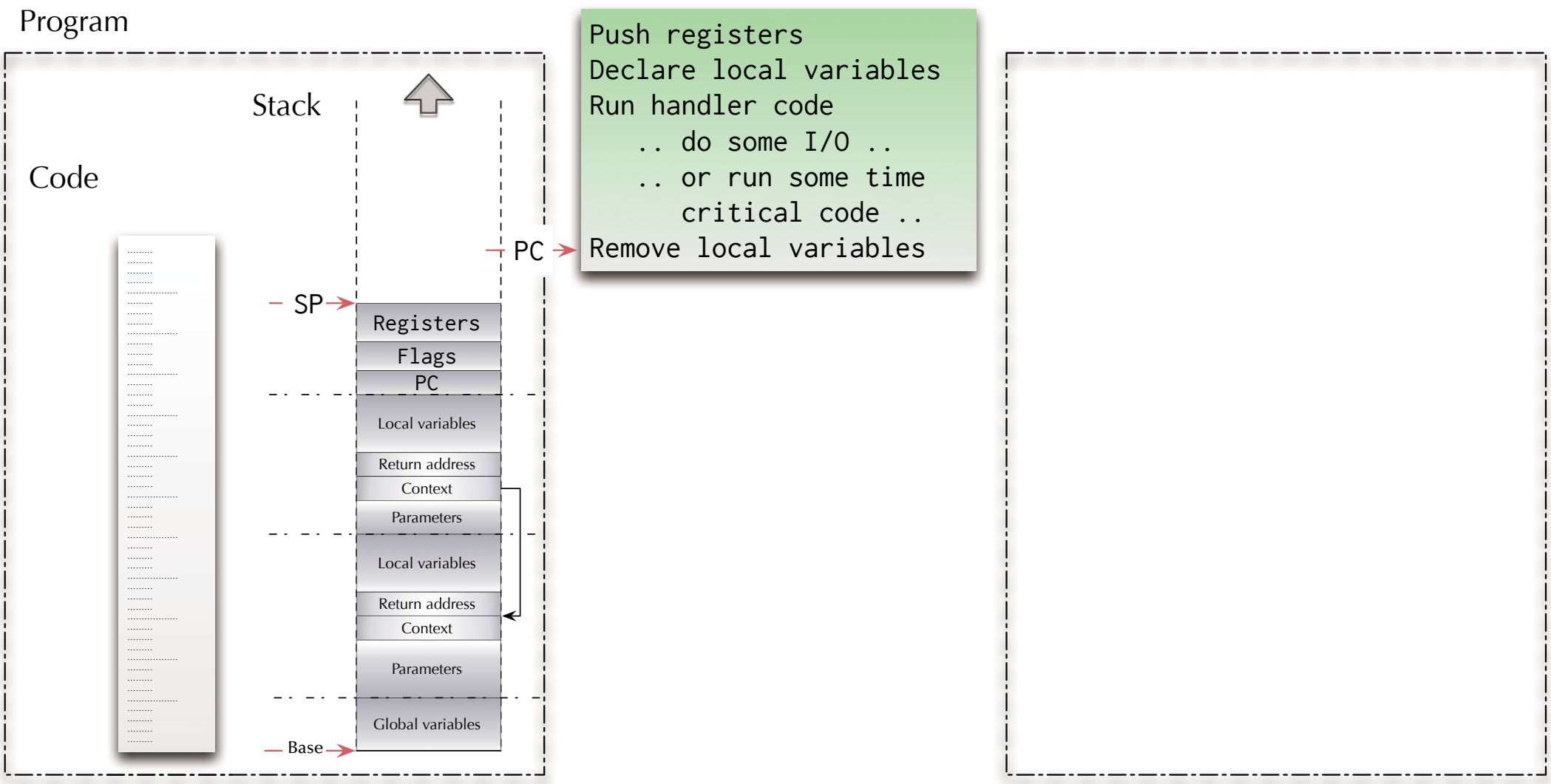
Interrupt processing



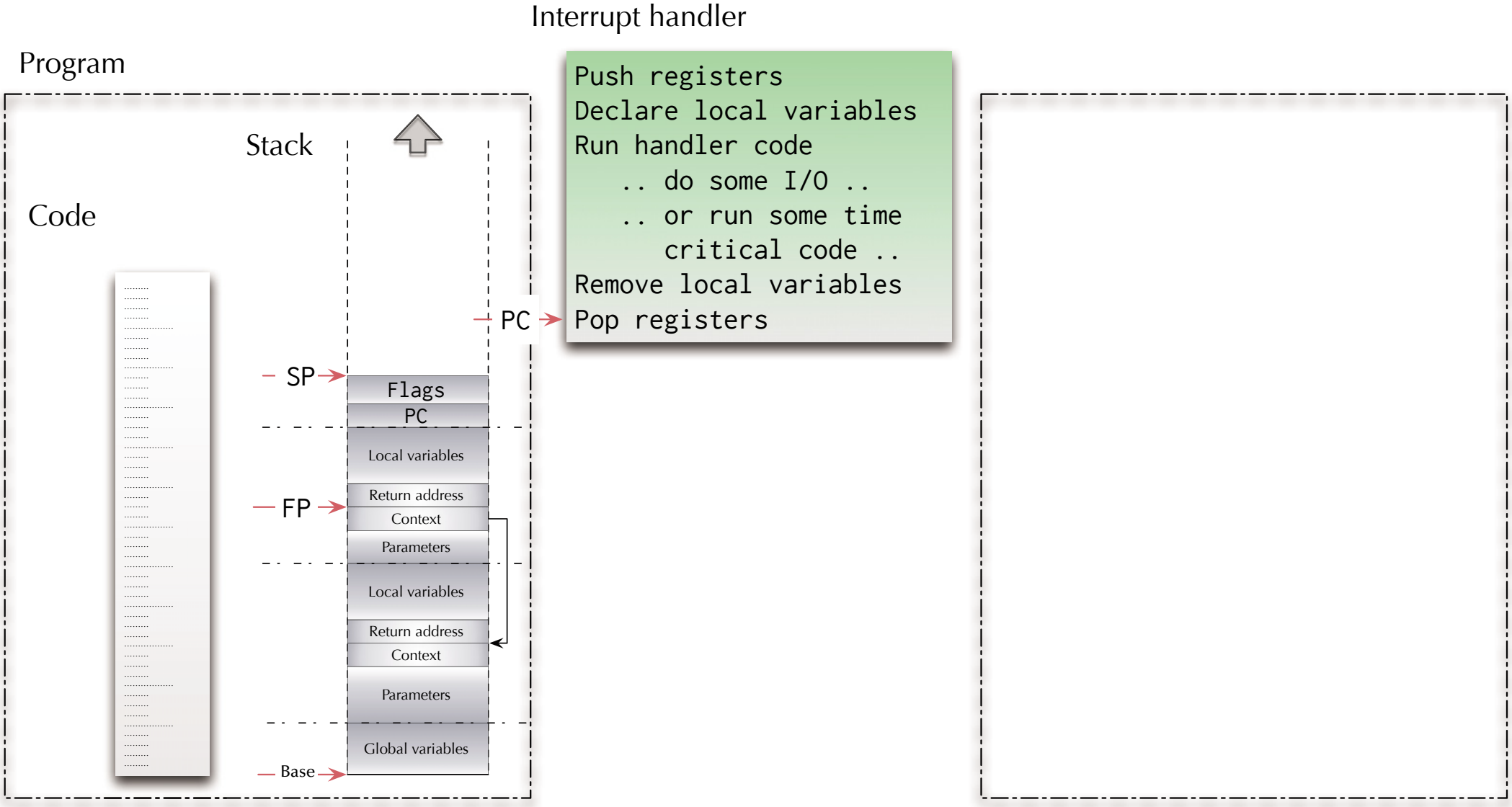
Interrupt processing



Interrupt processing

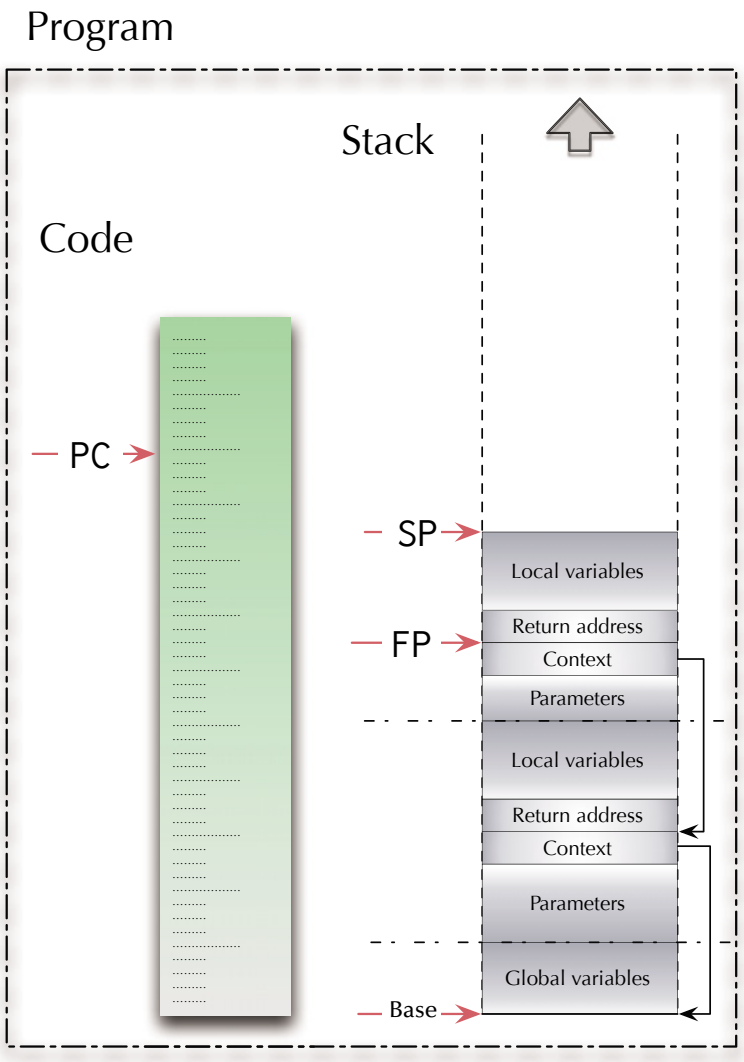


Interrupt processing



Interrupt processing

Interrupt handler

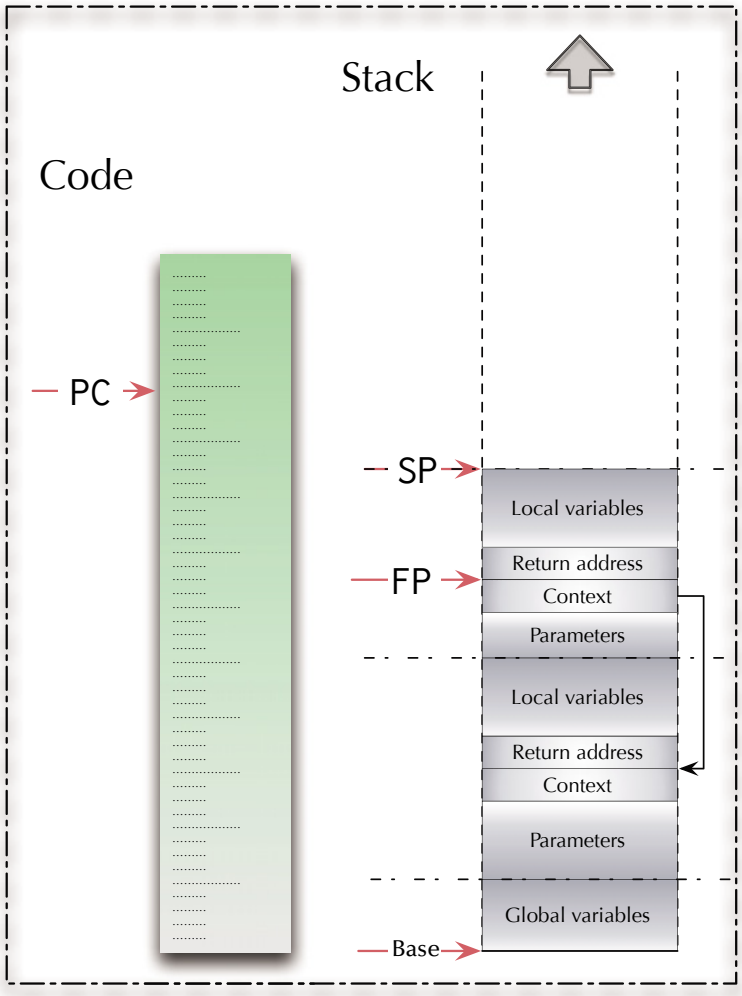



```
Push registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
Remove local variables
Pop registers
Return from interrupt
```

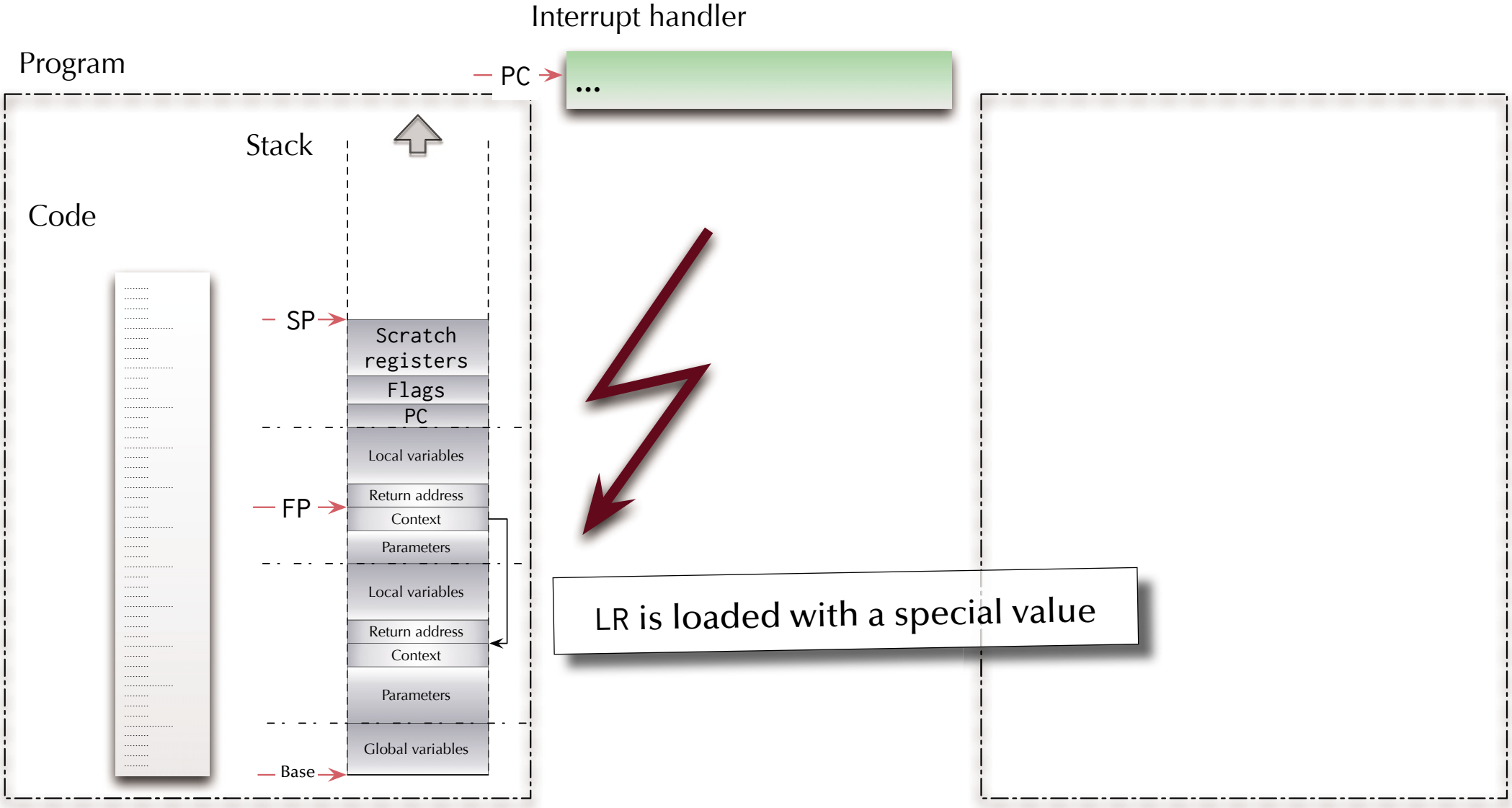
Interrupt processing

Interrupt handler

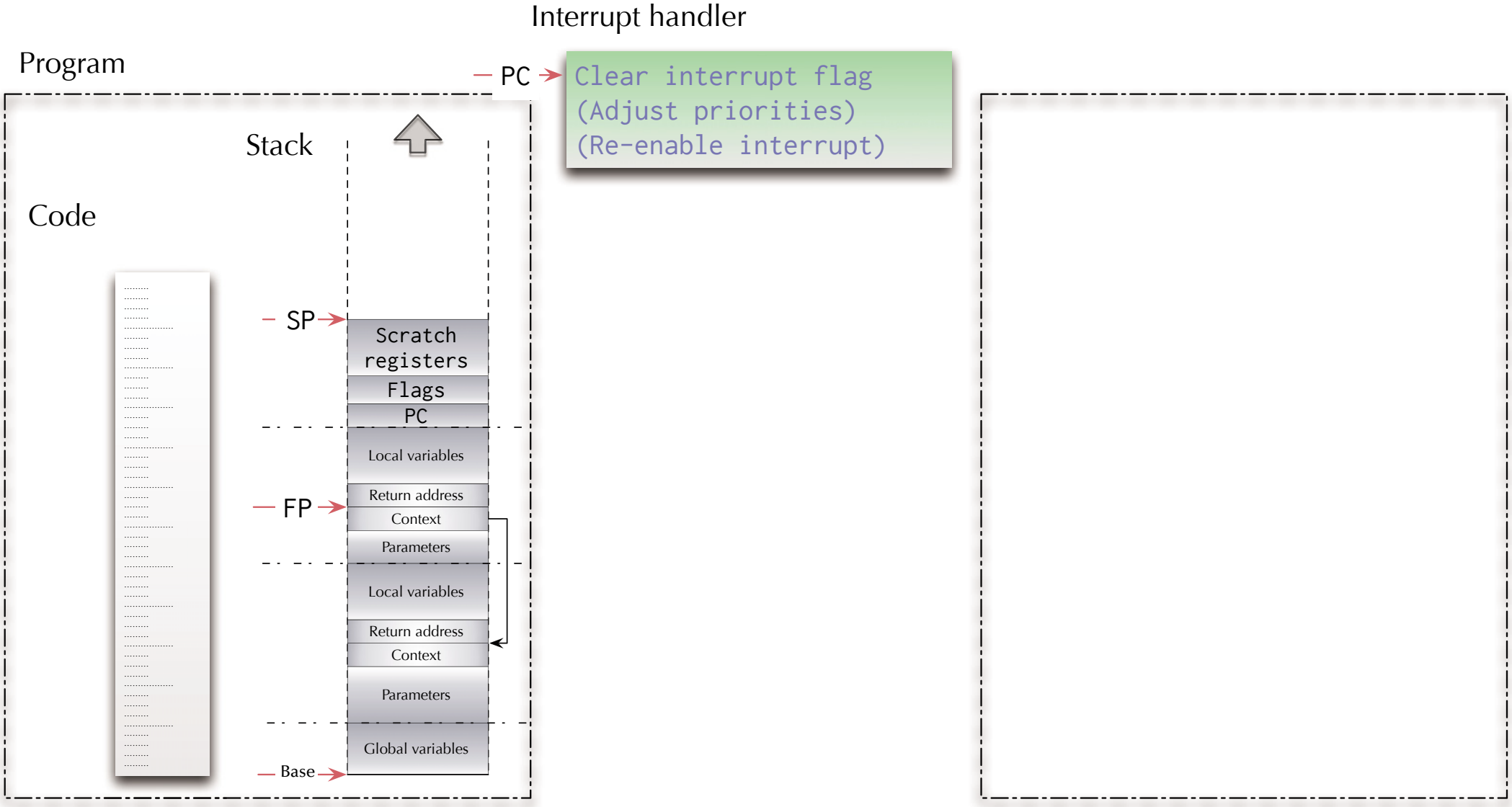
Program



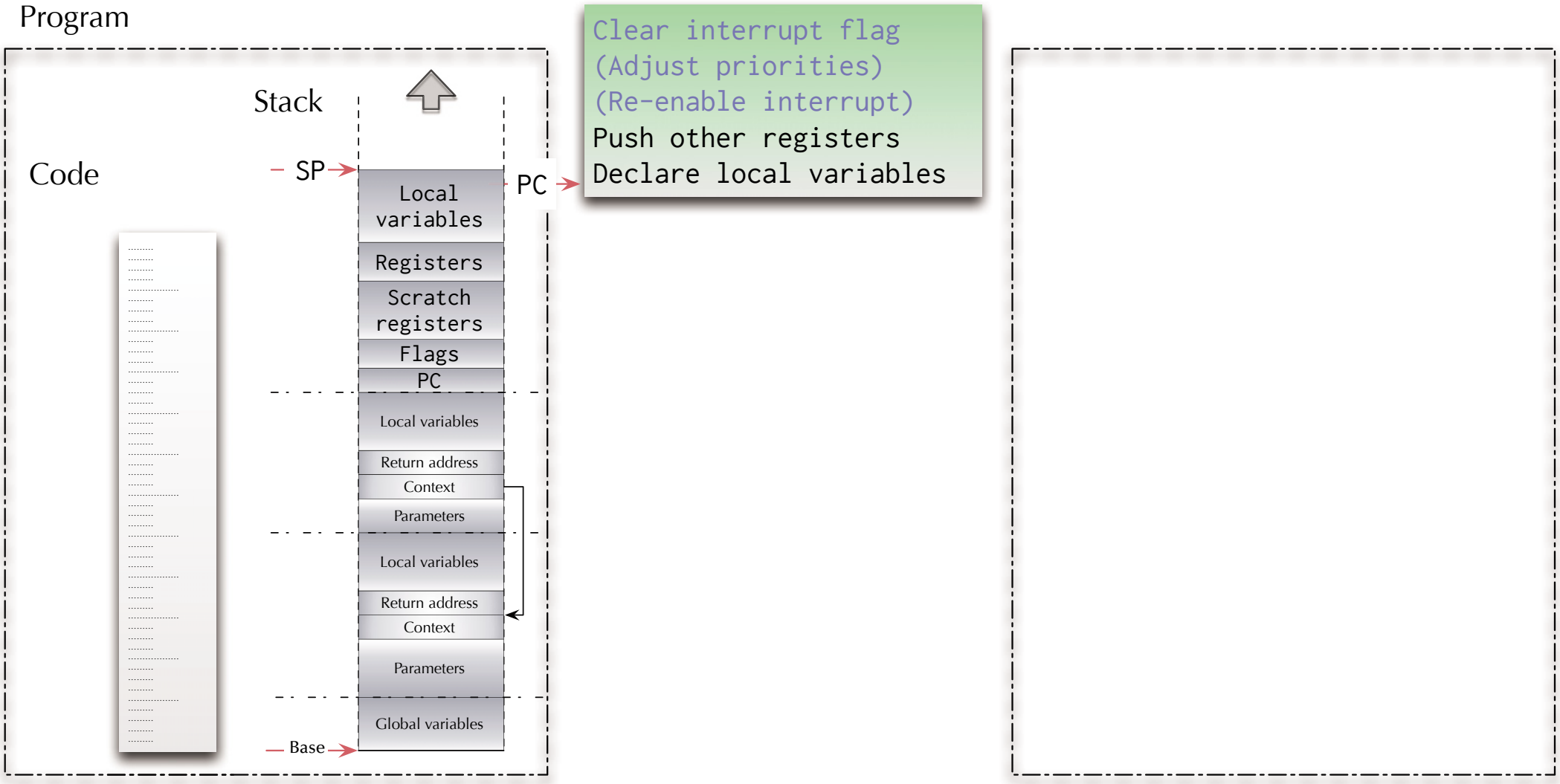
Interrupt processing



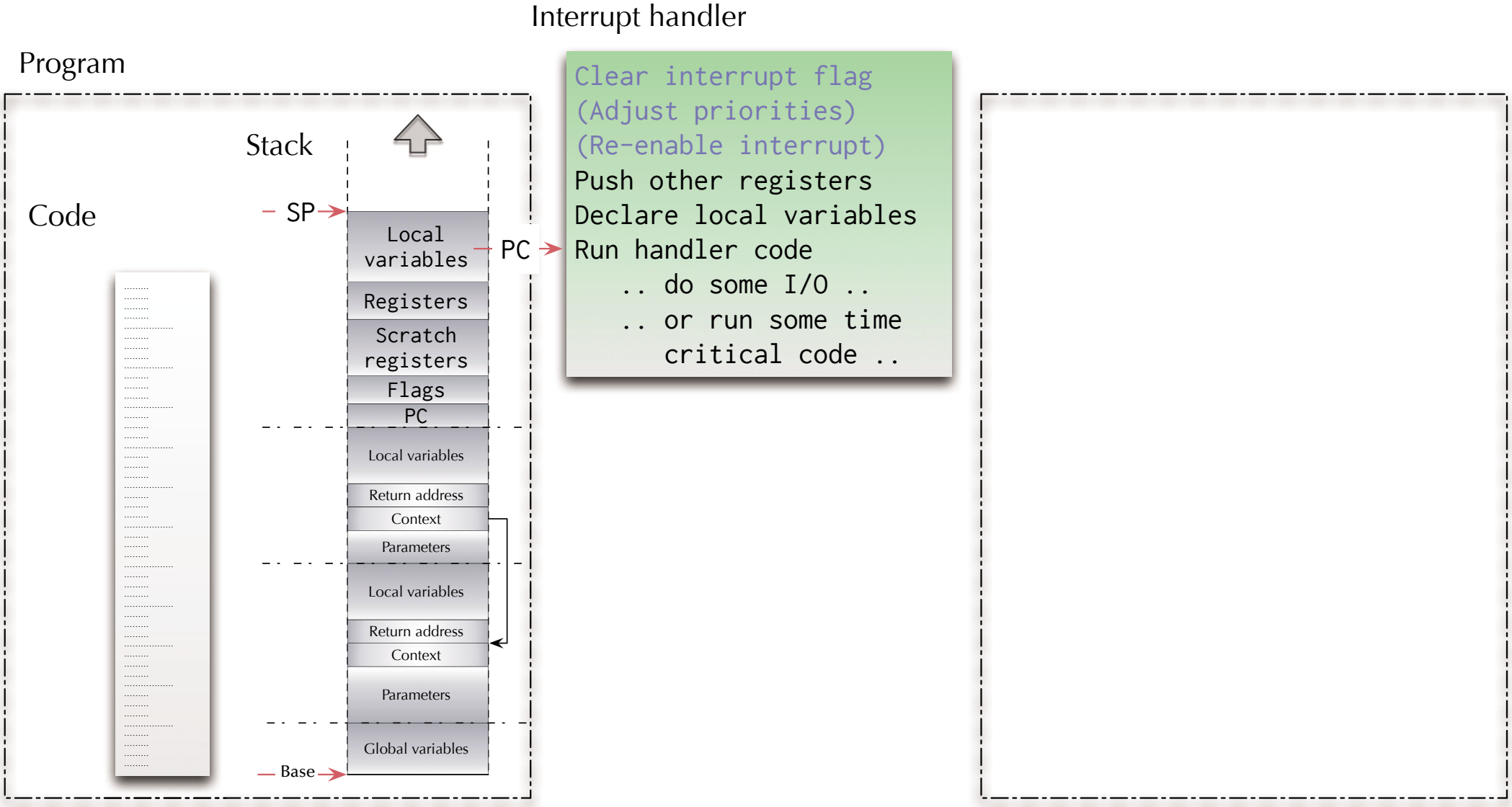
Interrupt processing



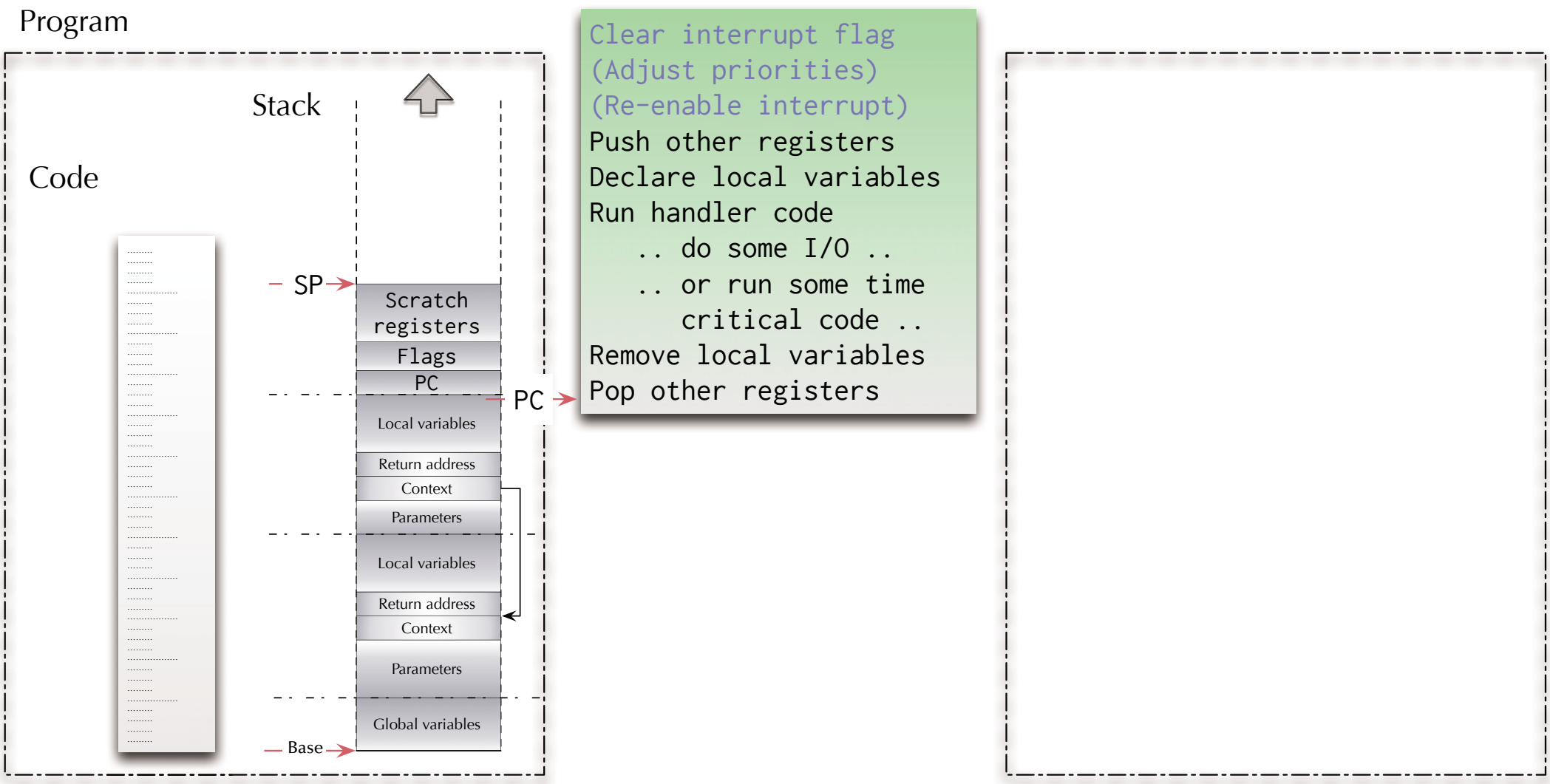
Interrupt processing



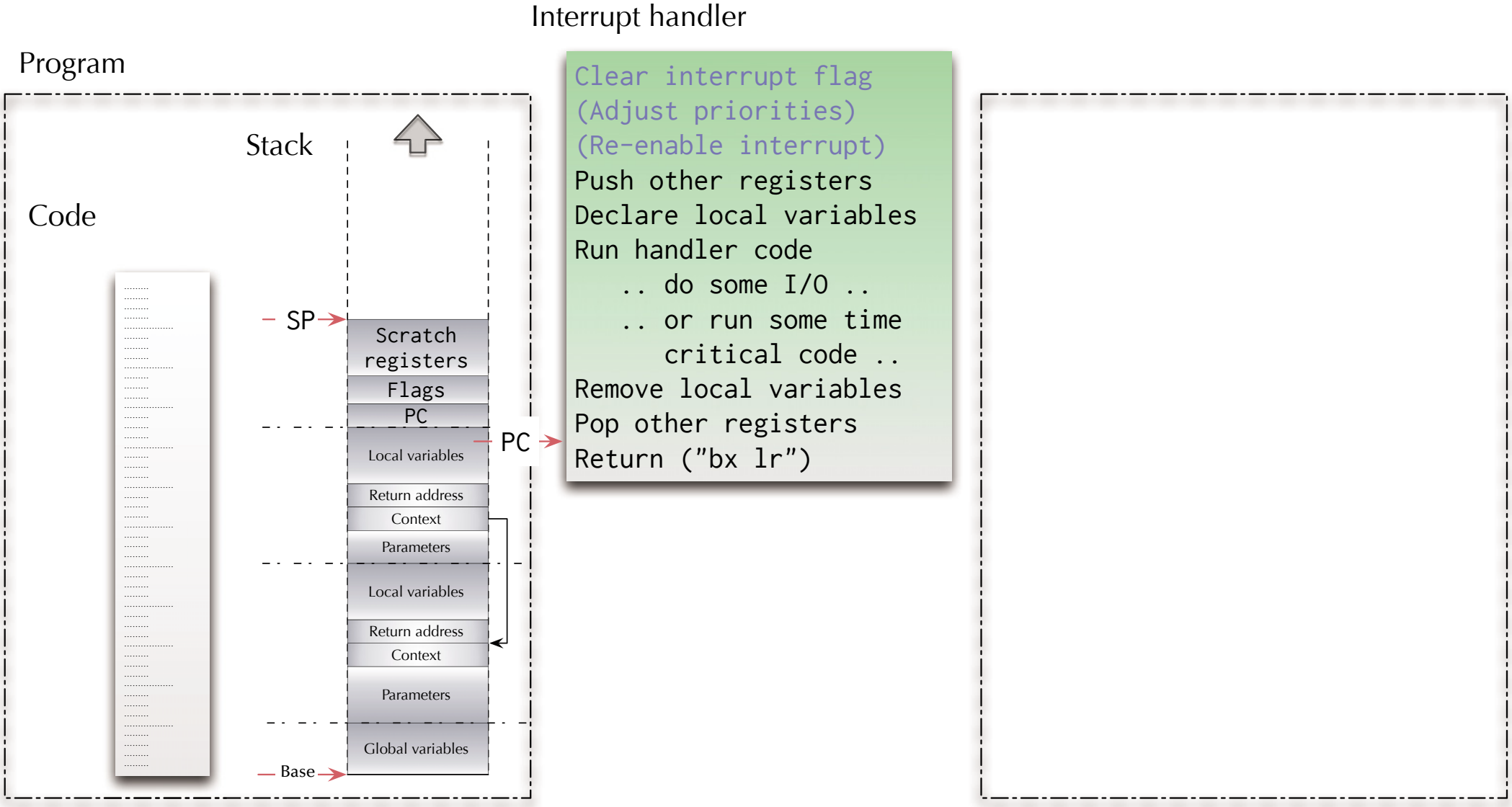
Interrupt processing



Interrupt processing



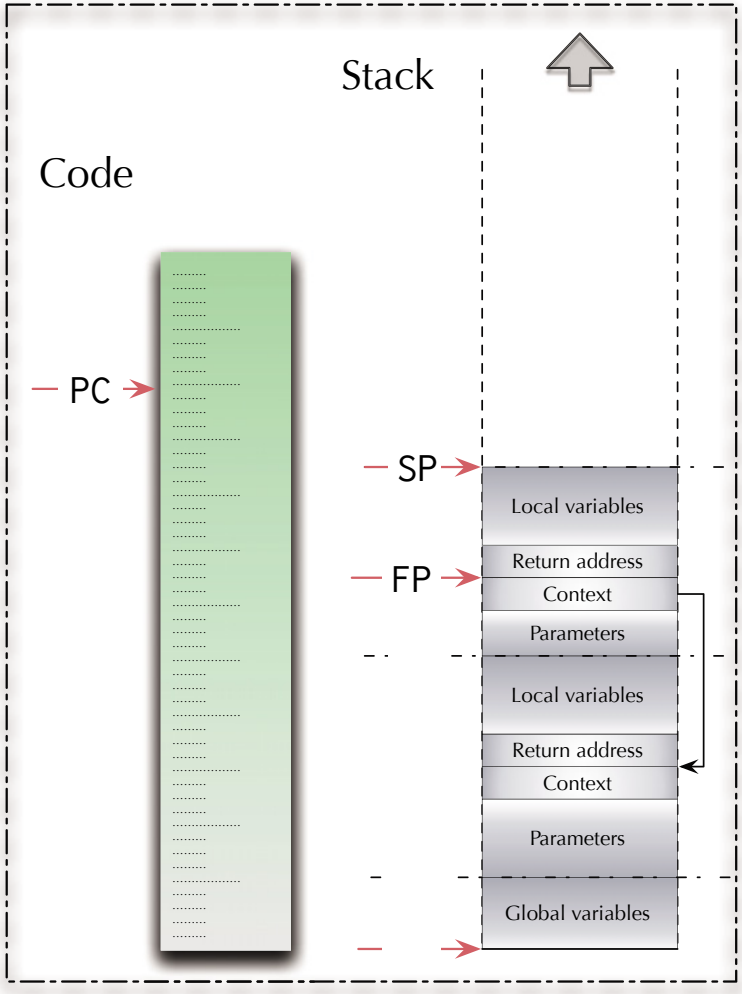
Interrupt processing



Interrupt processing

Interrupt handler

Program




```
Clear interrupt flag
(Adjust priorities)
(Re-enable interrupt)
Push other registers
Declare local variables
Run handler code
   .. do some I/O ..
   .. or run some time
      critical code ..
Remove local variables
Pop other registers
Return ("bx lr")
```

Interrupt handler

Things to consider

- ☞ Interrupt handler code can be interrupted as well.
- ☞ Are you allowing to interrupt an interrupt handler with an interrupt on the same priority level (e.g. the same interrupt)?
- ☞ Can you overrun a stack with interrupt handlers?

Interrupt handler

Things to consider

- Interrupt handler code can be interrupted as well.
- Are you allowing to interrupt an interrupt handler with an interrupt on the same priority level (e.g. the same interrupt)?
- Can you overrun a stack with interrupt handlers?
- Can we have one of those?

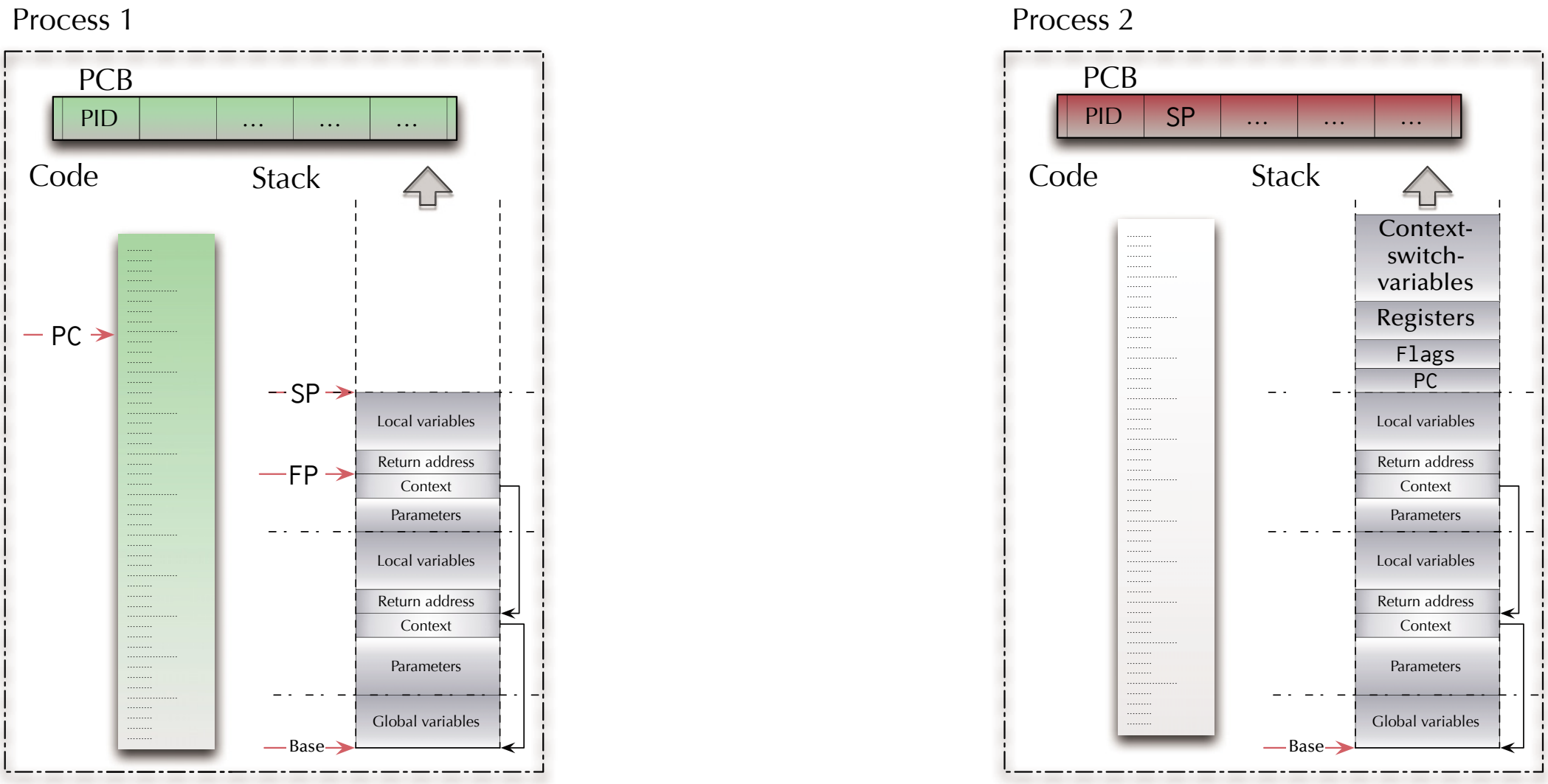
Multiple programs

If we can execute interrupt handler code
“concurrently” to our “main” program:

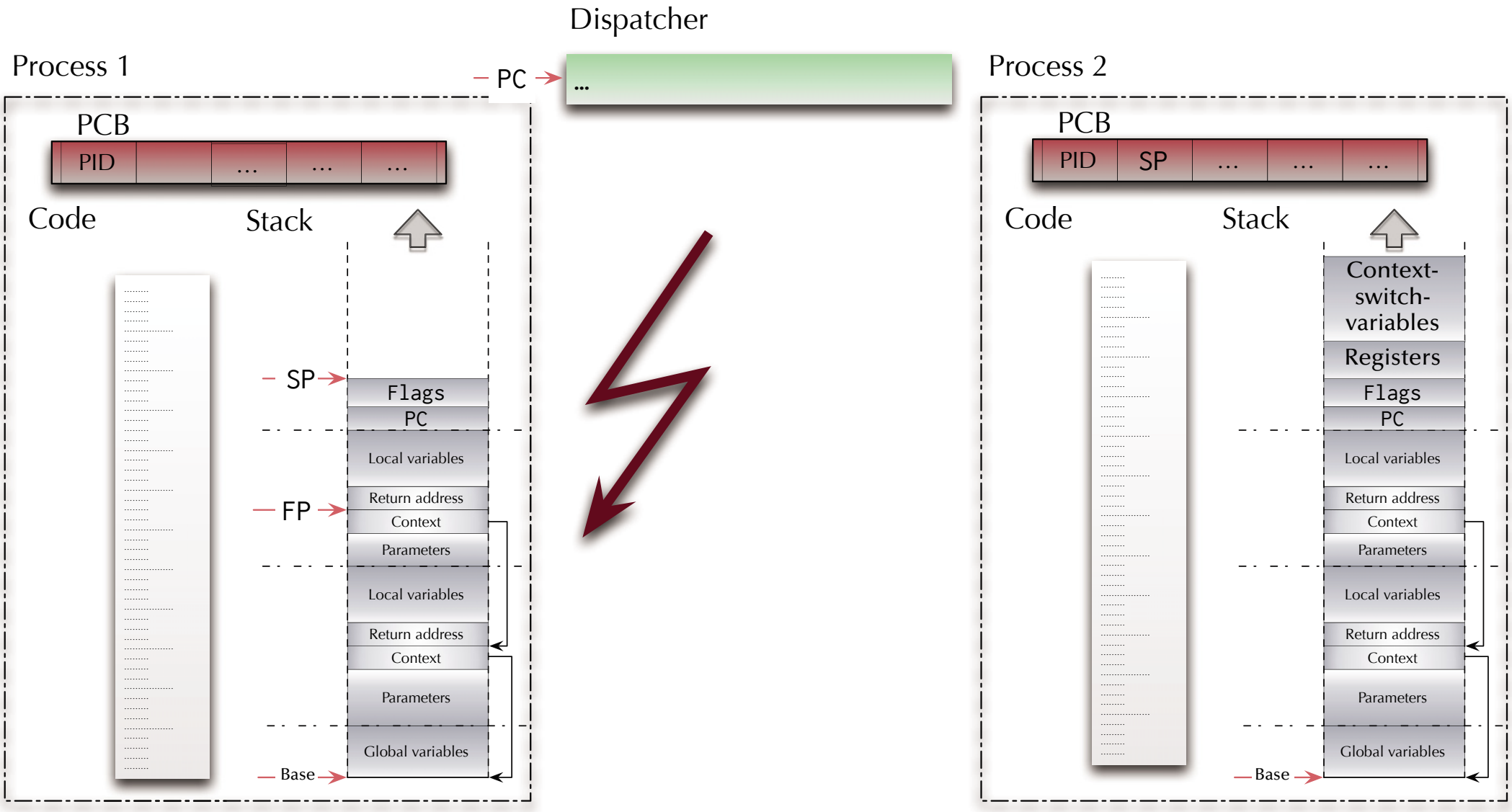
☞ Can we then also have multiple “main” programs?

Context switch

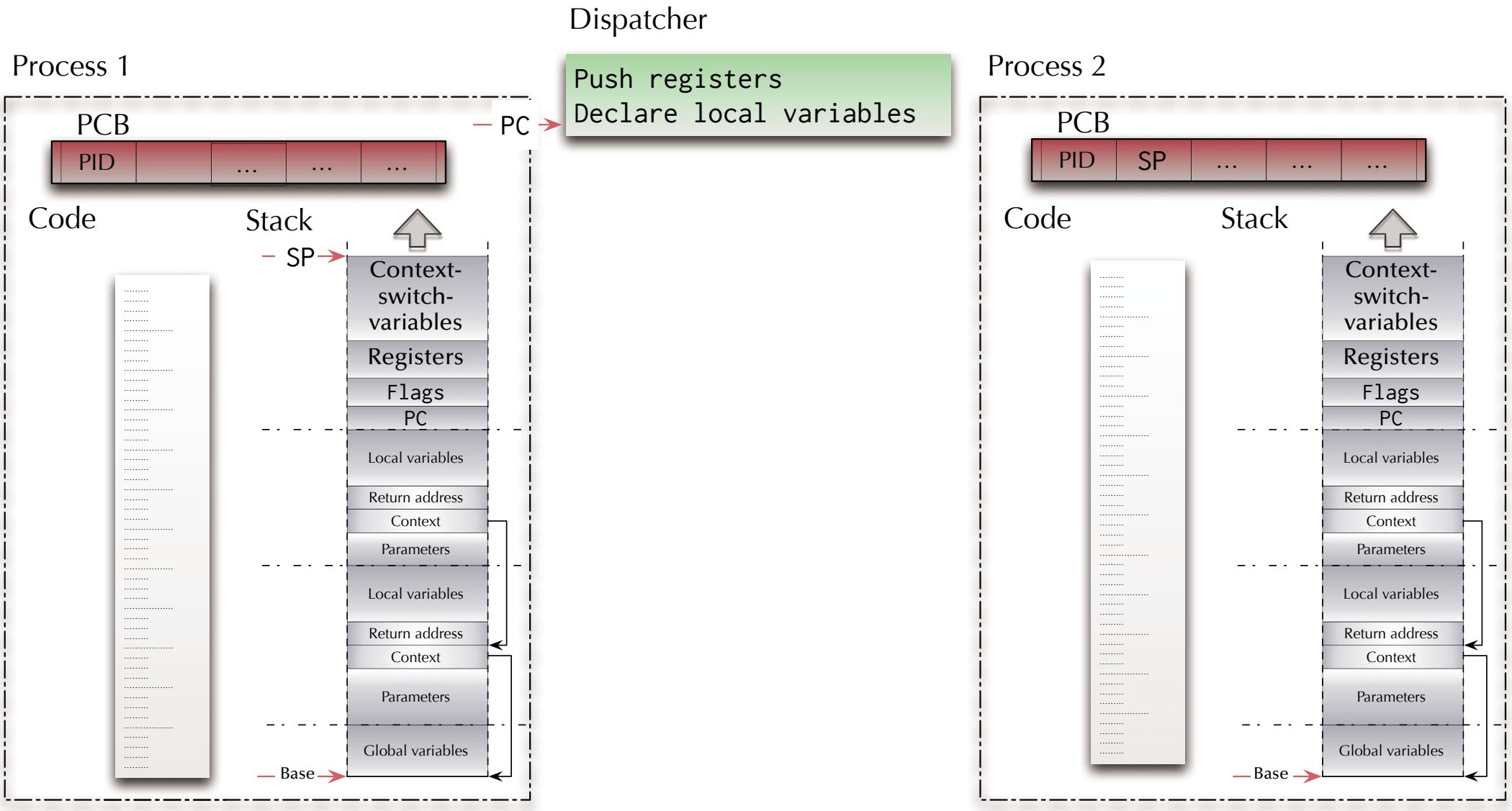
Dispatcher



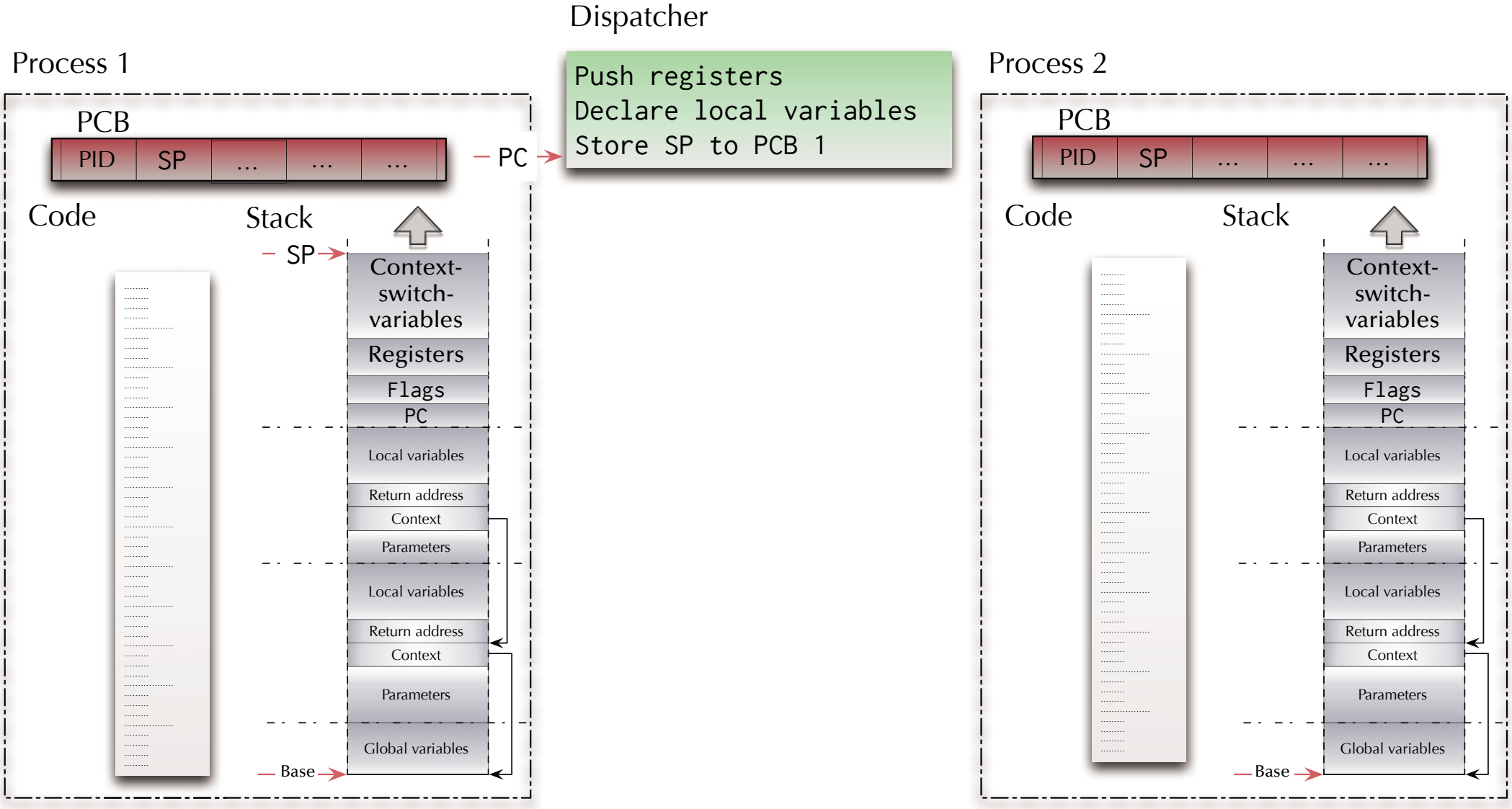
Context switch



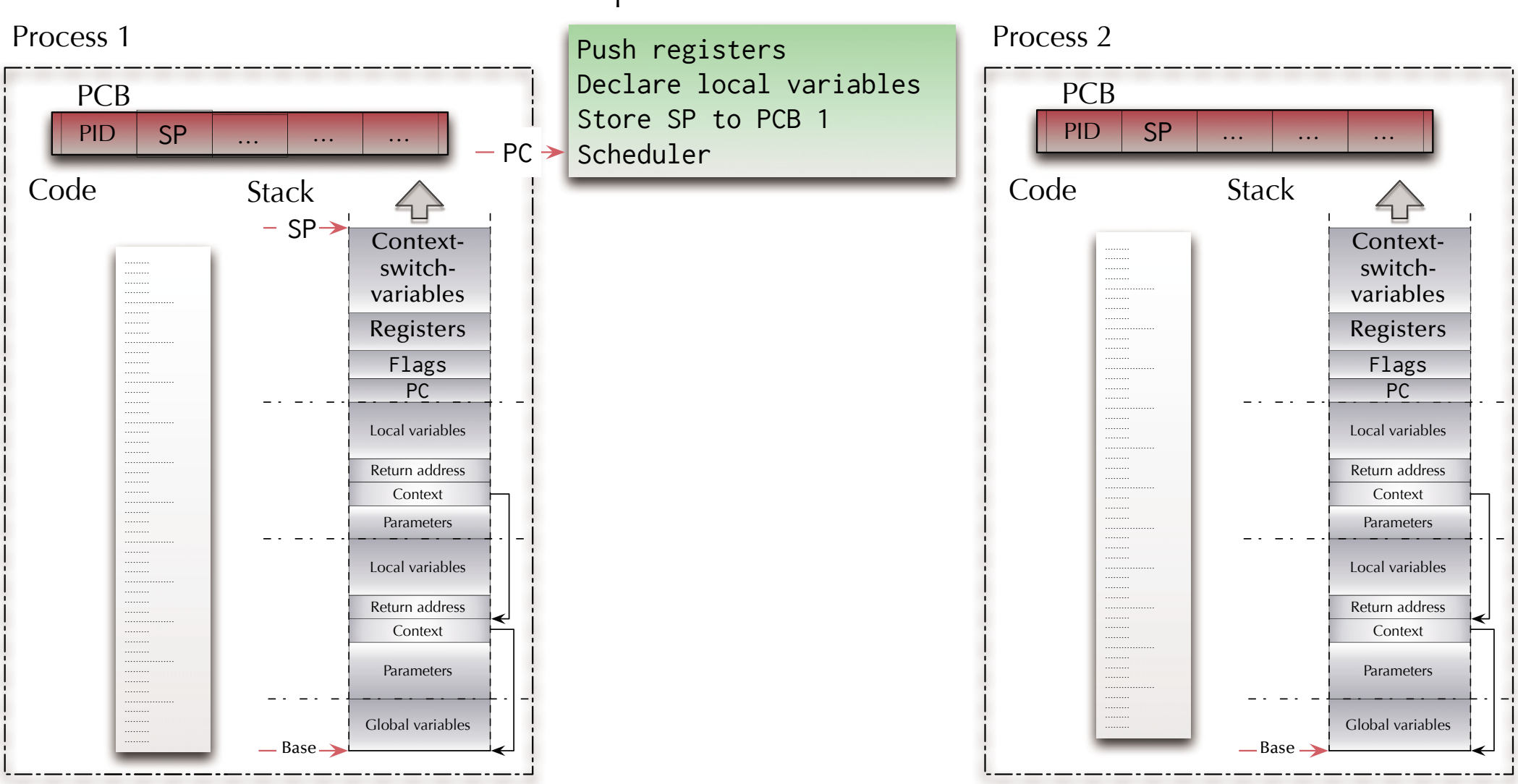
Context switch



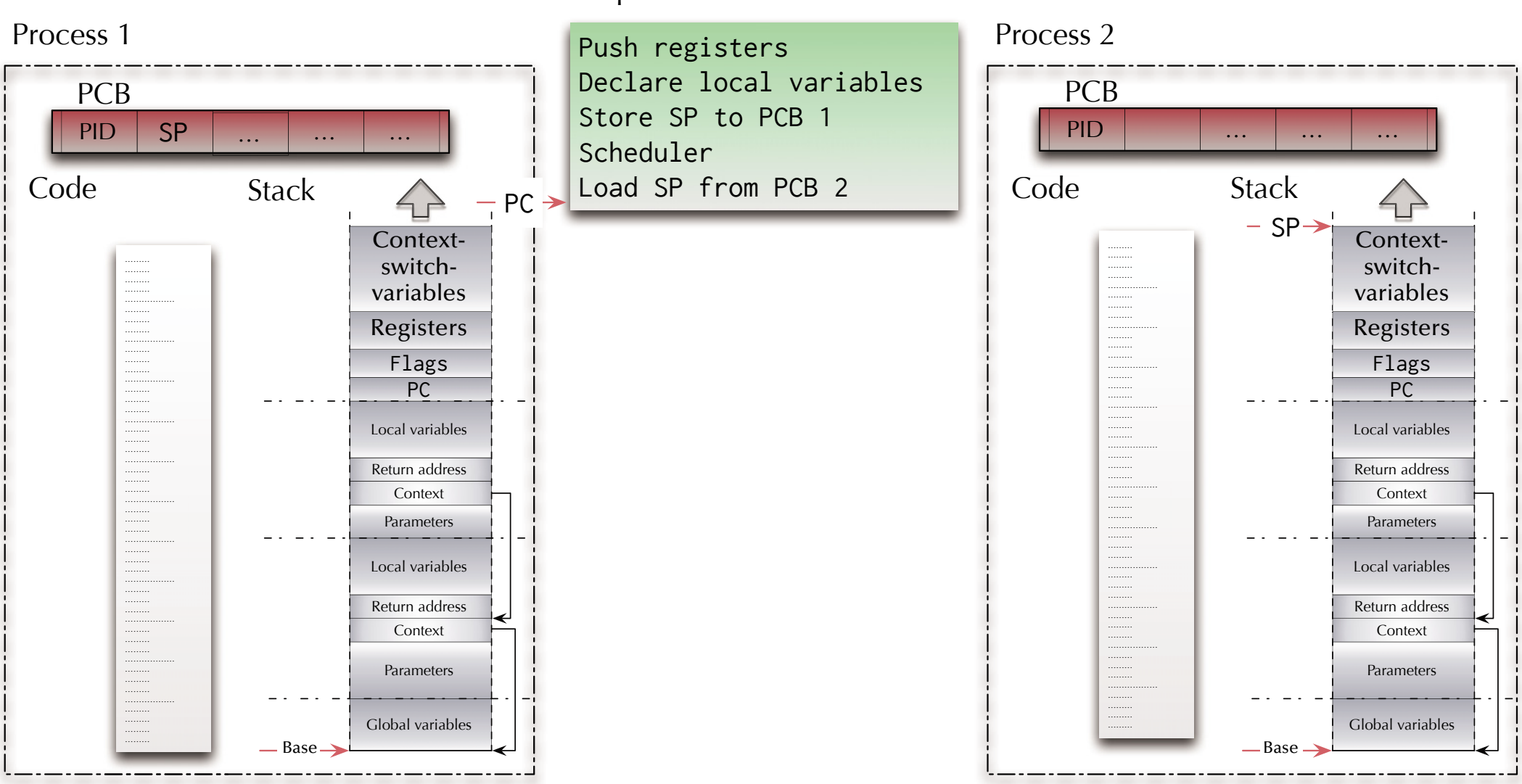
Context switch



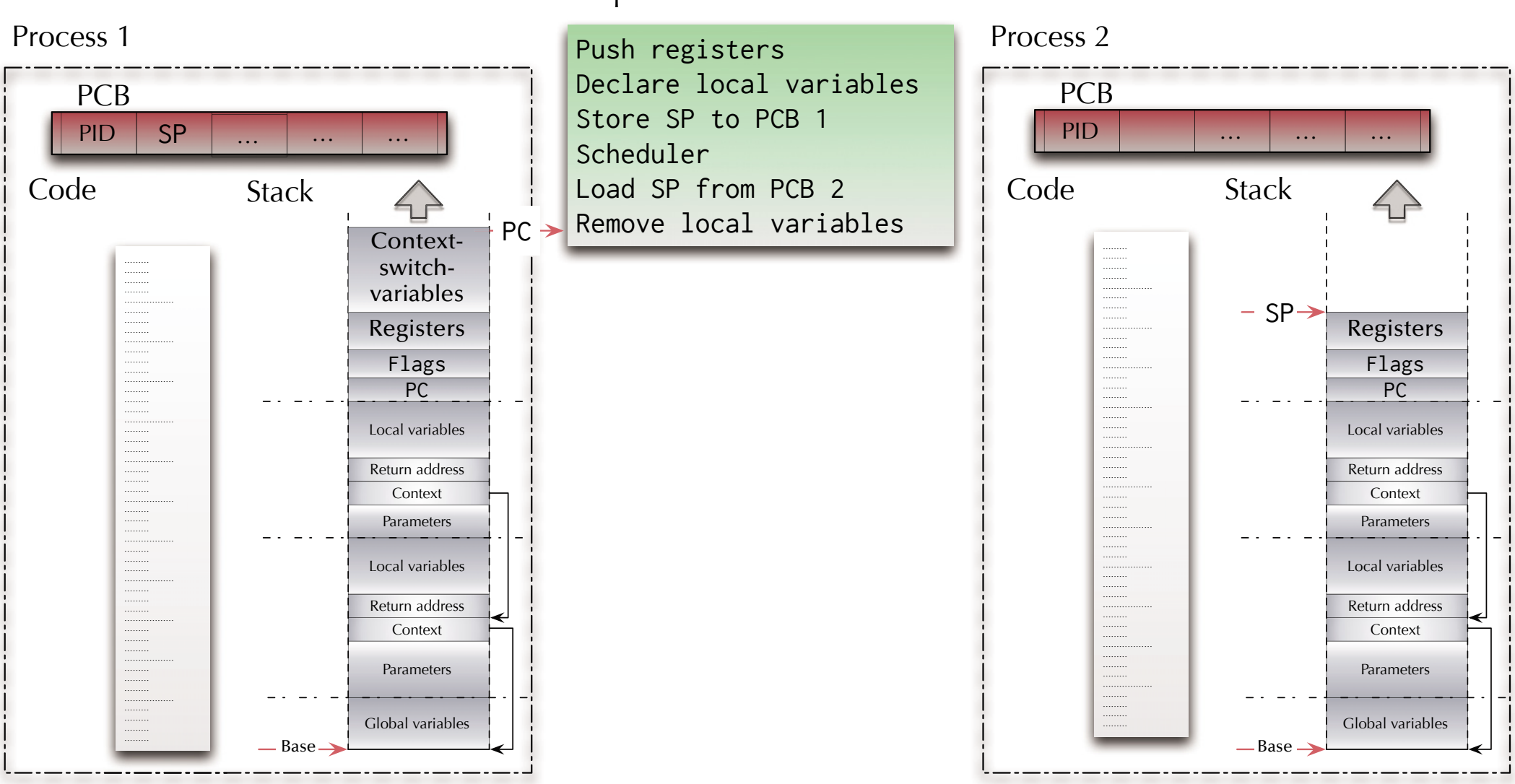
Context switch



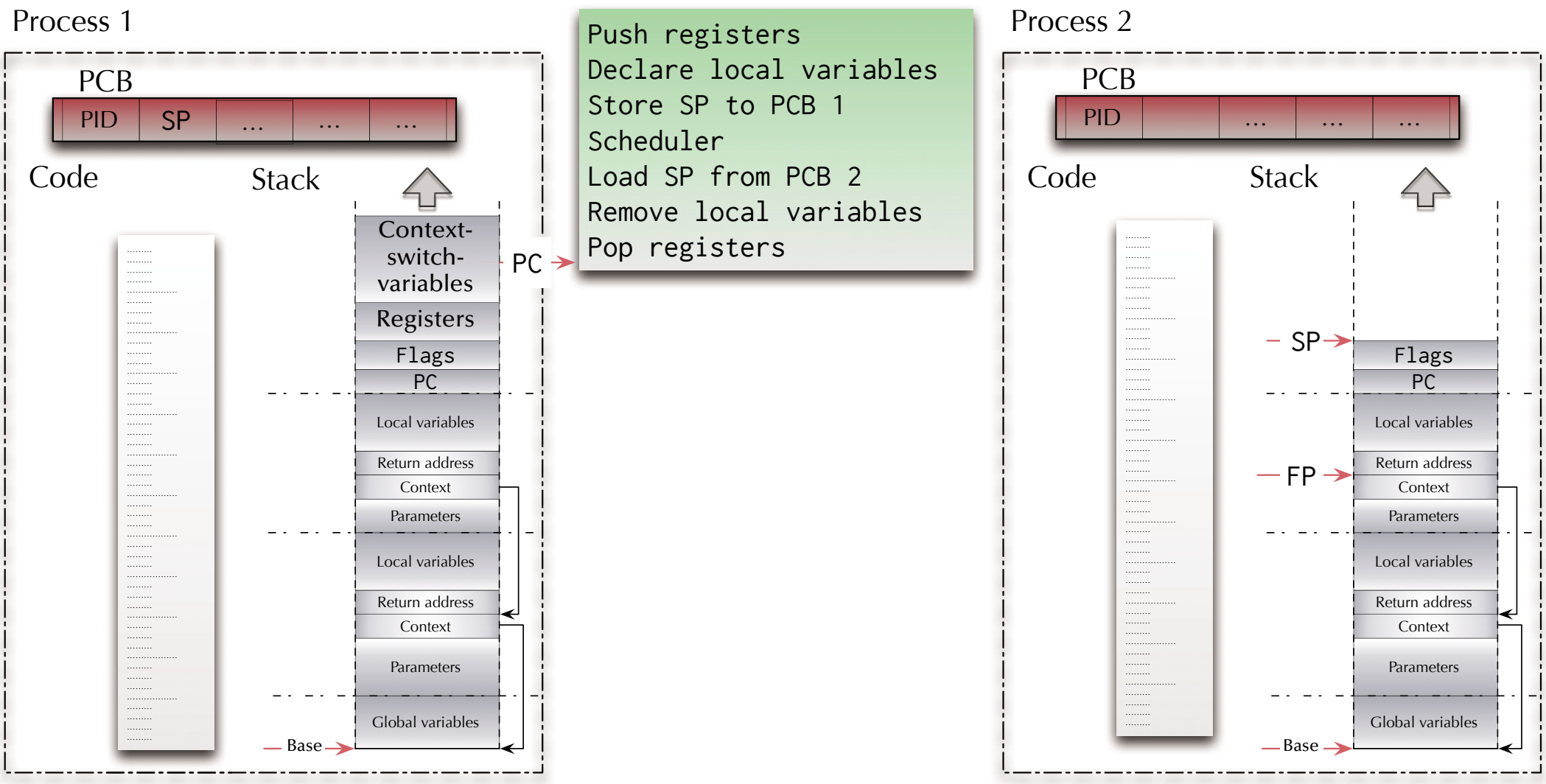
Context switch



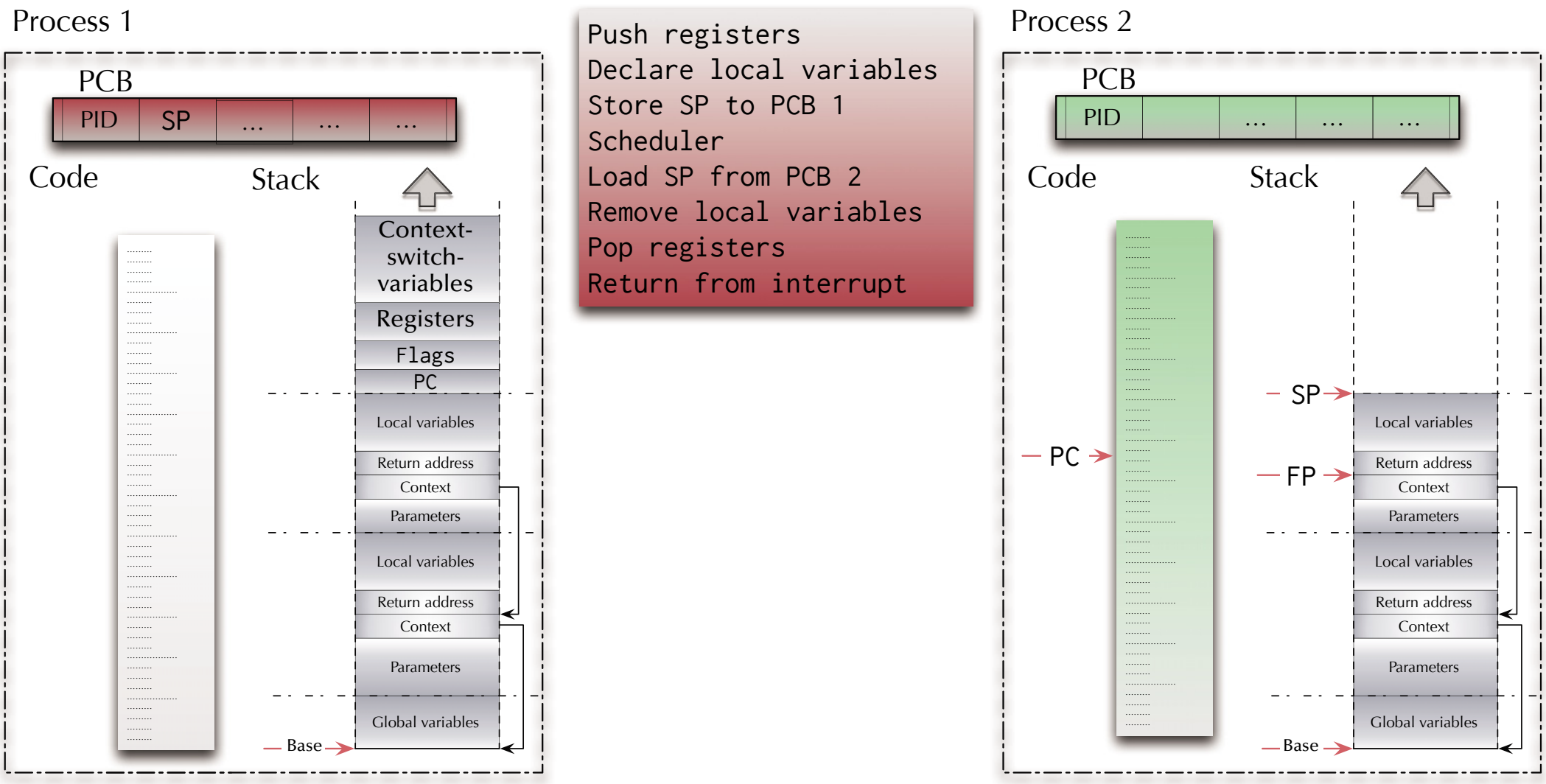
Context switch



Context switch



Context switch



Multi-tasking and Contention

Anything else could go wrong?

Multi-tasking and Contention

Anything else could go wrong?

☞ If there is neither communication nor contention between concurrent parts
… all is easy … and boring.

☞ What happens if concurrent programs **share** data?

Shared variables

Atomic load & store operations

☞ Assumption 1: every individual base memory cell (word) load and store access is *atomic*

☞ Assumption 2: there is *no* atomic combined load-store access

```
G : Natural := 0; -- assumed to be mapped on a 1-word cell in memory
```

```
task body P1 is
begin
   G := 1
   G := G + G;
end P1;
```

```
task body P2 is
begin
   G := 2
   G := G + G;
end P2;
```

```
task body P3 is
begin
   G := 3
   G := G + G;
end P3;
```

☞ What is the value of `G`?

Shared variables

Atomic load & store operations

☞ Assumption 1: every individual base memory cell (word) load and store access is *atomic*

☞ Assumption 2: there is *no* atomic combined load-store access

```
G: .word 0x00000000
```

```
ldr   r4, =G
mov   r1, #1
str   r1, [r4]
ldr   r2, [r4]
ldr   r3, [r4]
add   r1, r2, r3
str   r1, [r4]
```

```
ldr   r4, =G
mov   r1, #2
str   r1, [r4]
ldr   r2, [r4]
ldr   r3, [r4]
add   r1, r2, r3
str   r1, [r4]
```

```
ldr   r4, =G
mov   r1, #3
str   r1, [r4]
ldr   r2, [r4]
ldr   r3, [r4]
add   r1, r2, r3
str   r1, [r4]
```

☞ What is the value in memory cell `G` after all three programs complete?

Shared variables

This is terrible!

Nobody is their right mind would analyse a program like that.

☞ … are we missing something?

☞ … is there an elegant way out?

Mutual exclusion … or the lack thereof

```
Count : Integer := 0;
```

```
task body Enter is
begin
 for i := 1 .. 100 loop
  Count := Count + 1;
 end loop;
end Enter;
```

```
task body Leave is
begin
 for i := 1 .. 100 loop
  Count := Count - 1;
 end loop;
end Leave;
```

☞ What is the value of `Count` after both programs complete?

Mutual exclusion … or the lack thereof

```
Count: .word 0x00000000
```

```
   ldr   r4, =Count
   mov   r1, #1
for_enter:
   cmp   r1, #100
   bgt   end_for_enter
   ldr   r2, [r4]
   add   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_enter
end_for_enter:
```

```
   ldr   r4, =Count
   mov   r1, #1
for_leave:
   cmp   r1, #100
   bgt   end_for_leave
   ldr   r2, [r4]
   sub   r2, #1
   str   r2, [r4]
   add   r1, #1
   b     for_leave
end_for_leave:
```

☞ What is the value at address Count after both programs complete?

Mutual exclusion … or the lack thereof

```
Count: .word 0x00000000
```

```
    ldr   r4, =Count
    mov   r1, #1
for_enter:
    cmp   r1, #100
    bgt   end_for_enter
    ldr   r2, [r4]
    add   r2, #1
    str   r2, [r4]
    add   r1, #1
    b     for_enter
end_for_enter:
```

```
    ldr   r4, =Count
    mov   r1, #1
for_leave:
    cmp   r1, #100
    bgt   end_for_leave
    ldr   r2, [r4]
    sub   r2, #1
    str   r2, [r4]
    add   r1, #1
    b     for_leave
end_for_leave:
```

☞ What is the value at address Count after both programs complete?

Mutual exclusion … or the lack thereof

```
Count: .word 0x00000000
```

```
   ldr   r4, =Count
   mov   r1, #1
for_enter:
   cmp   r1, #100
   bgt   end_for_enter
enter_strex_fail:
   ldrex r2, [r4] ; tag [r4] as exclusive
   add   r2, #1
   strex r2, [r4] ; only if untouched
   cbnz  r2, enter_strex_fail
   add   r1, #1
   b     for_enter
end_for_enter:
```

```
   ldr   r4, =Count
   mov   r1, #1
for_leave:
   cmp   r1, #100
   bgt   end_for_leave
leave_strex_fail:
   ldrex r2, [r4] ; tag [r4] as exclusive
   sub   r2, #1
   strex r2, [r4] ; only if untouched
   cbnz  r2, leave_strex_fail
   add   r1, #1
   b     for_leave
end_for_leave:
```

☞ What is the value at address Count after both programs complete?

Mutual exclusion

```
Count: .word 0x00000000
```

```
    ldr   r4, =Count
    mov   r1, #1
for_enter:
    cmp   r1, #100
    bgt   end_for_enter
enter_strex_fail:
    ldrex r2, [r4] ; tag [r4] as exclusive
    add   r2, #1
    strex r2, [r4] ; only if untouched
    cbnz  r2, enter_strex_fail
    add   r1, #1
    b     for_enter
end_for_enter:
```

```
    ldr   r4, =Count
    mov   r1, #1
for_leave:
    cmp   r1, #100
    bgt   end_for_leave
leave_strex_fail:
    ldrex r2, [r4] ; tag [r4] as exclusive
    sub   r2, #1
    strex r2, [r4] ; only if untouched
    cbnz  r2, leave_strex_fail
    add   r1, #1
    b     for_leave
end_for_leave:
```

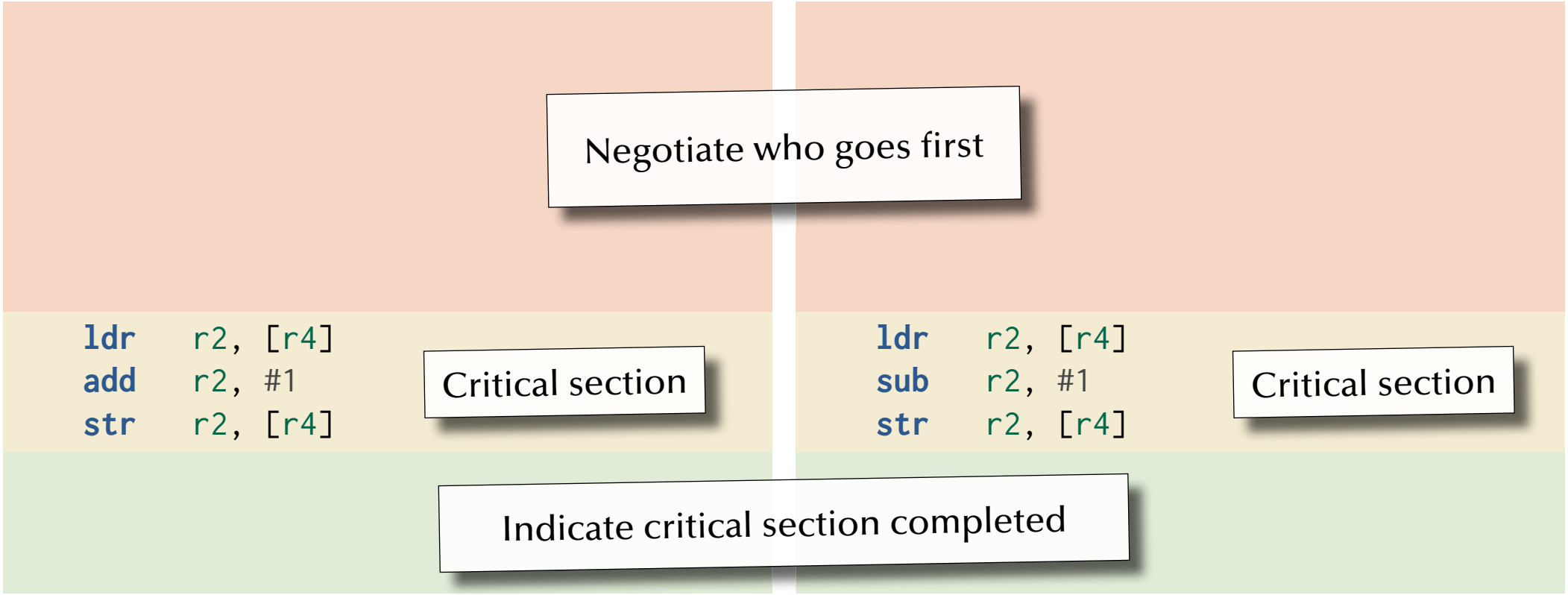
Any context switch needs to clear reservations

☞ What is the value at address Count after both programs complete?

```
Count:    .word 0x00000000
```

```
    ldr   r4, =Count
    mov   r1, #1
for_enter:
    cmp   r1, #100
    bgt   end_for_enter
```

```
    ldr   r4, =Count
    mov   r1, #1
for_leave:
    cmp   r1, #100
    bgt   end_for_leave
```




```
    ldr   r2, [r4]
    add   r2, #1
    str   r2, [r4]
```

```
    ldr   r2, [r4]
    sub   r2, #1
    str   r2, [r4]
```

```
    add   r1, #1
    b     for_enter
end_for_enter:
```

```
    add   r1, #1
    b     for_leave
end_for_leave:
```

```
Count:    .word 0x00000000
Lock:     .word 0x00000000 ; #0 means unlocked

   ldr    r3, =Lock
   ldr    r4, =Count
   mov    r1, #1
for_enter:
   cmp    r1, #100
   bgt    end_for_enter
fail_enter:
   ldr    r0, [r3]
   cbnz   r0, fail_enter  ; if locked
   ldr    r2, [r4]
   add    r2, #1
   str    r2, [r4]
   add    r1, #1
   b      for_enter
end_for_enter:
```

Critical section

```
   ldr    r3, =Lock
   ldr    r4, =Count
   mov    r1, #1
for_leave:
   cmp    r1, #100
   bgt    end_for_leave
fail_leave:
   ldr    r0, [r3]
   cbnz   r0, fail_leave  ; if locked
   ldr    r2, [r4]
   sub    r2, #1
   str    r2, [r4]
   add    r1, #1
   b      for_leave
end_for_leave:
```

Critical section

```
Count:    .word 0x00000000
Lock:     .word 0x00000000 ; #0 means unlocked

   ldr    r3, =Lock
   ldr    r4, =Count
   mov    r1, #1
for_enter:
   cmp    r1, #100
   bgt    end_for_enter
fail_enter:
   ldr    r0, [r3]
   cbnz   r0, fail_enter   ; if locked
   mov    r0, #1           ; lock value
   str    r0, [r3]         ; lock

   ldr    r2, [r4]
   add    r2, #1
   str    r2, [r4]

   add    r1, #1
   b      for_enter
end_for_enter:
```

Critical section

```
   ldr    r3, =Lock
   ldr    r4, =Count
   mov    r1, #1
for_leave:
   cmp    r1, #100
   bgt    end_for_leave
fail_leave:
   ldr    r0, [r3]
   cbnz   r0, fail_leave   ; if locked
   mov    r0, #1           ; lock value
   str    r0, [r3]         ; lock

   ldr    r2, [r4]
   sub    r2, #1
   str    r2, [r4]

   add    r1, #1
   b      for_leave
end_for_leave:
```

Critical section

```
Count:    .word 0x00000000
Lock:     .word 0x00000000 ; #0 means unlocked

   ldr    r3, =Lock
   ldr    r4, =Count
   mov    r1, #1
for_enter:
   cmp    r1, #100
   bgt    end_for_enter
fail_enter:
   ldrex  r0, [r3]
   cbnz   r0, fail_enter   ; if locked
   mov    r0, #1           ; lock value
   strex  r0, [r3]         ; try lock
   cbnz   r0, fail_enter   ; if touched
   dmb                     ; sync memory
   ldr    r2, [r4]
   add    r2, #1
   str    r2, [r4]
   add    r1, #1
   b      for_enter
end_for_enter:
```

```
   ldr    r3, =Lock
   ldr    r4, =Count
   mov    r1, #1
for_leave:
   cmp    r1, #100
   bgt    end_for_leave
fail_leave:
   ldrex  r0, [r3]
   cbnz   r0, fail_leave   ; if locked
   mov    r0, #1           ; lock value
   strex  r0, [r3]         ; try lock
   cbnz   r0, fail_leave   ; if touched
   dmb                     ; sync memory
   ldr    r2, [r4]
   sub    r2, #1
   str    r2, [r4]
   add    r1, #1
   b      for_leave
end_for_leave:
```

Any context switch needs to clear reservations

Critical section

Critical section

```
Count:    .word 0x00000000
Lock:     .word 0x00000000 ; #0 means unlocked

   ldr    r3, =Lock
   ldr    r4, =Count
   mov    r1, #1
for_enter:
   cmp    r1, #100
   bgt    end_for_enter
fail_enter:
   ldrex  r0, [r3]
   cbnz   r0, fail_enter   ; if locked
   mov    r0, #1           ; lock value
   strex  r0, [r3]         ; try lock
   cbnz   r0, fail_enter   ; if touched
   dmb                     ; sync memory
   ldr    r2, [r4]
   add    r2, #1
   str    r2, [r4]
   dmb                     ; sync memory
   mov    r0, #0           ; unlock value
   str    r0, [r3]         ; unlock
   add    r1, #1
   b      for_enter
end_for_enter:
```

Any context switch needs to clear reservations

Critical section

```
   ldr    r3, =Lock
   ldr    r4, =Count
   mov    r1, #1
for_leave:
   cmp    r1, #100
   bgt    end_for_leave
fail_leave:
   ldrex  r0, [r3]
   cbnz   r0, fail_leave   ; if locked
   mov    r0, #1           ; lock value
   strex  r0, [r3]         ; try lock
   cbnz   r0, fail_leave   ; if touched
   dmb                     ; sync memory
   ldr    r2, [r4]
   sub    r2, #1
   str    r2, [r4]
   dmb                     ; sync memory
   mov    r0, #0           ; unlock value
   str    r0, [r3]         ; unlock
   add    r1, #1
   b      for_leave
end_for_leave:
```

Critical section

Mutual exclusion: atomic test-and-set operation

```
type Flag is Natural range 0..1; C : Flag := 0;
```

```
task body Pi is
L : Flag;
begin
 loop
  loop
   [L := C; C := 1];
   exit when L = 0;
   ------ change process
  end loop;
  ------ critical_section_i;
  C := 0;
 end loop;
end Pi;
```

```
task body Pj is
L : Flag;
begin
 loop
  loop
   [L := C; C := 1];
   exit when L = 0;
   ------ change process
  end loop;
  ------ critical_section_j;
  C := 0;
 end loop;
end Pj;
```

☞ Does that work?

Mutual exclusion: atomic test-and-set operation

```
type Flag is Natural range 0..1; C : Flag := 0;
```

```
task body Pi is
L : Flag;
begin
 loop
  loop
   [L := C; C := 1];
   exit when L = 0;
   ------ change process
  end loop;
  ------ critical_section_i;
  C := 0;
 end loop;
end Pi;
```

```
task body Pj is
L : Flag;
begin
 loop
  loop
   [L := C; C := 1];
   exit when L = 0;
   ------ change process
  end loop;
  ------ critical_section_j;
  C := 0;
 end loop;
end Pj;
```

☞ Mutual exclusion!, No deadlock!, No global live-lock!

☞ Works for any dynamic number of processes.

☞ Individual starvation possible! Busy waiting loops!

Mutual exclusion: atomic exchange operation

```
type Flag is Natural range 0..1; C : Flag := 0;
```

```
task body Pi is
L : Flag := 1;
begin
 loop
  loop
   [Temp := L; L := C; C := Temp];
   exit when L = 0;
   ------ change process
  end loop;
  ------ critical_section_i;
  L := 1; C := 0;
 end loop;
end Pi;
```

```
task body Pj is
L : Flag := 1;
begin
 loop
  loop
   [Temp := L; L := C; C := Temp];
   exit when L = 0;
   ------ change process
  end loop;
  ------ critical_section_j;
  L := 1; C := 0;
 end loop;
end Pj;
```

☞ Does that work?

Mutual exclusion: atomic exchange operation

```
type Flag is Natural range 0..1; C : Flag := 0;
```

```
task body Pi is
L : Flag := 1;
begin
 loop
  loop
   [Temp := L; L := C; C := Temp];
   exit when L = 0;
   ------ change process
  end loop;
  ------ critical_section_i;
  L := 1; C := 0;
 end loop;
end Pi;
```

```
task body Pj is
L : Flag := 1;
begin
 loop
  loop
   [Temp := L; L := C; C := Temp];
   exit when L = 0;
   ------ change process
  end loop;
  ------ critical_section_j;
  L := 1; C := 0;
 end loop;
end Pj;
```

- ☞ Mutual exclusion!, No deadlock!, No global live-lock!
- ☞ Works for any dynamic number of processes.
- ☞ Individual starvation possible! Busy waiting loops!

Mutual exclusion: memory cell reservation

```
type Flag is Natural range 0..1; C : Flag := 0;
```

```
task body Pi is
L : Flag;
begin
 loop
  loop
   L :R= C; C :T= 1;
   exit when Untouched and L = 0;
   ------ change process
  end loop;
  ------ critical_section_i;
  C := 0;
 end loop;
end Pi;
```

```
task body Pj is
L : Flag;
begin
 loop
  loop
   L :R= C; C :T= 1;
   exit when Untouched and L = 0;
   ------ change process
  end loop;
  ------ critical_section_j;
  C := 0;
 end loop;
end Pj;
```

☞ Does that work?

Mutual exclusion: memory cell reservation

```
type Flag is Natural range 0..1; C : Flag := 0;
```

Any context switch needs to clear reservations

```
task body Pi is
L : Flag;
begin
 loop
  loop
   L :R= C; C :T= 1;
   exit when Untouched and L = 0;
   ------ change process
  end loop;
  ------ critical_section_i;
  C := 0;
 end loop;
end Pi;
```

```
task body Pj is
L : Flag;
begin
 loop
  loop
   L :R= C; C :T= 1;
   exit when Untouched and L = 0;
   ------ change process
  end loop;
  ------ critical_section_j;
  C := 0;
 end loop;
end Pj;
```

☞ Mutual exclusion!, No deadlock!, No global live-lock!

☞ Works for any dynamic number of processes.

☞ Individual starvation possible! Busy waiting loops!

```
Count:    .word 0x00000000
Lock:     .word 0x00000000 ; #0 means unlocked

   ldr    r3, =Lock
   ldr    r4, =Count
   mov    r1, #1
for_enter:
   cmp    r1, #100
   bgt    end_for_enter
fail_enter:
   ldrex  r0, [r3]
   cbnz   r0, fail_enter   ; if locked
   mov    r0, #1           ; lock value
   strex  r0, [r3]         ; try lock
   cbnz   r0, fail_enter   ; if touched
   dmb                     ; sync memory
   ldr    r2, [r4]
   add    r2, #1
   str    r2, [r4]
   dmb                     ; sync memory
   mov    r0, #0           ; unlock value
   str    r0, [r3]         ; unlock
   add    r1, #1
   b      for_enter
end_for_enter:
```

Any context switch needs to clear reservations

Critical section

```
   ldr    r3, =Lock
   ldr    r4, =Count
   mov    r1, #1
for_leave:
   cmp    r1, #100
   bgt    end_for_leave
fail_leave:
   ldrex  r0, [r3]
   cbnz   r0, fail_leave   ; if locked
   mov    r0, #1           ; lock value
   strex  r0, [r3]         ; try lock
   cbnz   r0, fail_leave   ; if touched
   dmb                     ; sync memory
   ldr    r2, [r4]
   sub    r2, #1
   str    r2, [r4]
   dmb                     ; sync memory
   mov    r0, #0           ; unlock value
   str    r0, [r3]         ; unlock
   add    r1, #1
   b      for_leave
end_for_leave:
```

Asks for permission

Critical section

Mutual exclusion

```
Count: .word 0x00000000

    ldr   r4, =Count
    mov   r1, #1
for_enter:
    cmp   r1, #100
    bgt   end_for_enter
enter_strex_fail:
    ldrex r2, [r4] ; tag [r4] as exclusive
    add   r2, #1
    strex r2, [r4] ; only if untouched
    cbnz  r2, enter_strex_fail
    add   r1, #1
    b     for_enter
end_for_enter:
```

```
    ldr   r4, =Count
    mov   r1, #1
for_leave:
    cmp   r1, #100
    bgt   end_for_leave
leave_strex_fail:
    ldrex r2, [r4] ; tag [r4] as exclusive
    sub   r2, #1
    strex r2, [r4] ; only if untouched
    cbnz  r2, leave_strex_fail
    add   r1, #1
    b     for_leave
end_for_leave:
```

Any context switch needs to clear reservations

Asks for forgiveness

☞ Light weight solution – sometimes referred to as “lock-free” or “lockless”.

Beyond atomic hardware operations

Semaphores

Basic definition (Dijkstra 1968)

Assuming the following three conditions on a shared memory cell between processes:

- a set of processes agree on a variable **S** operating as a flag to indicate synchronization conditions
- an atomic operation **P** on S — for 'passeren' (Dutch for 'pass'):

  P(S): `[as soon as S > 0 then S := S - 1]` ☞ this is a potentially delaying operation
- an atomic operation **V** on S — for 'vrygeven' (Dutch for 'to release'):

  V(S): `[S := S + 1]`

☞ *then* the variable **S** is called a **Semaphore**.

Beyond atomic hardware operations

Semaphores

… as supplied by operating systems and runtime environments

- a set of processes $P_1 \ldots P_N$ agree on a variable **S** operating as a flag to indicate synchronization conditions

- an atomic operation **Wait** on S: (aka 'Suspend_Until_True', 'sem_wait', …)

  Process P_i : **Wait** (S):

  ```
  [if S > 0 then S := S - 1
            else suspend Pi on S]
  ```

- an atomic operation **Signal** on S: (aka 'Set_True', 'sem_post', …)

  Process P_i : **Signal** (S):

  ```
  [if ∃ Pj suspended on S then release Pj
                          else S := S + 1]
  ```

☞ *then* the variable **S** is called a **Semaphore** in a scheduling environment.

Semaphores

Types of semaphores:

- **Binary semaphores**: restricted to [0, 1] or [False, True] resp.
  Multiple `V` (`Signal`) calls have the same effect than a single call.
  - Atomic hardware operations support binary semaphores.
  - Binary semaphores are sufficient to create all other semaphore forms.
- **General semaphores** (counting semaphores): non-negative number; (range limited by the system) `P` and `V` increment and decrement the semaphore by one.
- **Quantity semaphores**: The increment (and decrement) value for the semaphore is specified as a parameter with P and V.

☞ All types of semaphores must be initialized:
often the number of processes which are allowed inside a critical section, i.e. '1'.

```
Count:    .word 0x00000000
Sema:     .word 0x00000001

   ldr    r3, =Sema
   ldr    r4, =Count
   mov    r1, #1
for_enter:
   cmp    r1, #100
   bgt    end_for_enter
wait_1:
   ldr    r0, [r3]
   cbz    r0, wait_1 ; if Semaphore = 0
   …                                    Critical section
   add    r1, #1
   b      for_enter
end_for_enter:
```

```
   ldr    r3, =Sema
   ldr    r4, =Count
   mov    r1, #1
for_leave:
   cmp    r1, #100
   bgt    end_for_leave
wait_2:
   ldr    r0, [r3]
   cbz    r0, wait_2 ; if Semaphore = 0
   …                                    Critical section
   add    r1, #1
   b      for_leave
end_for_leave:
```

```
Count:    .word 0x00000000
Sema:     .word 0x00000001

    ldr   r3, =Sema
    ldr   r4, =Count
    mov   r1, #1
for_enter:
    cmp   r1, #100
    bgt   end_for_enter
wait_1:
    ldr   r0, [r3]
    cbz   r0, wait_1 ; if Semaphore = 0
    sub   r0, #1     ; dec Semaphore
    str   r0, [r3]   ; update

    …                                   Critical section

    add   r1, #1
    b     for_enter
end_for_enter:
```

```
    ldr   r3, =Sema
    ldr   r4, =Count
    mov   r1, #1
for_leave:
    cmp   r1, #100
    bgt   end_for_leave
wait_2:
    ldr   r0, [r3]
    cbz   r0, wait_2 ; if Semaphore = 0
    sub   r0, #1     ; dec Semaphore
    str   r0, [r3]   ; update

    …                                   Critical section

    add   r1, #1
    b     for_leave
end_for_leave:
```

```
Count:    .word 0x00000000
Sema:     .word 0x00000001

   ldr    r3, =Sema
   ldr    r4, =Count
   mov    r1, #1
for_enter:
   cmp    r1, #100
   bgt    end_for_enter
wait_1:
   ldrex  r0, [r3]
   cbz    r0, wait_1 ; if Semaphore = 0
   sub    r0, #1     ; dec Semaphore
   strex  r0, [r3]   ; try update
   cbnz   r0, wait_1 ; if touched
   dmb               ; sync memory
   …
   add    r1, #1
   b      for_enter
end_for_enter:
```

```
   ldr    r3, =Sema
   ldr    r4, =Count
   mov    r1, #1
for_leave:
   cmp    r1, #100
   bgt    end_for_leave
wait_2:
   ldrex  r0, [r3]
   cbz    r0, wait_2 ; if Semaphore = 0
   sub    r0, #1     ; dec Semaphore
   strex  r0, [r3]   ; try update
   cbnz   r0, wait_2 ; if touched
   dmb               ; sync memory
   …
   add    r1, #1
   b      for_leave
end_for_leave:
```

Any context switch needs to clear reservations

Critical section

Critical section

```
Count:    .word 0x00000000
Sema:     .word 0x00000001

   ldr    r3, =Sema
   ldr    r4, =Count
   mov    r1, #1
for_enter:
   cmp    r1, #100
   bgt    end_for_enter
wait_1:
   ldrex  r0, [r3]
   cbz    r0, wait_1 ; if Semaphore = 0
   sub    r0, #1     ; dec Semaphore
   strex  r0, [r3]   ; try update
   cbnz   r0, wait_1 ; if touched
   dmb               ; sync memory
   …

   ldr    r0, [r3]
   add    r0, #1         ; inc Semaphore
   str    r0, [r3]       ; update


   add    r1, #1
   b      for_enter
end_for_enter:
```

```
   ldr    r3, =Sema
   ldr    r4, =Count
   mov    r1, #1
for_leave:
   cmp    r1, #100
   bgt    end_for_leave
wait_2:
   ldrex  r0, [r3]
   cbz    r0, wait_2 ; if Semaphore = 0
   sub    r0, #1     ; dec Semaphore
   strex  r0, [r3]   ; try update
   cbnz   r0, wait_2 ; if touched
   dmb               ; sync memory
   …

   ldr    r0, [r3]
   add    r0, #1         ; inc Semaphore
   str    r0, [r3]       ; update


   add    r1, #1
   b      for_leave
end_for_leave:
```

Any context switch needs to clear reservations

Critical section

Critical section

```
Count:    .word 0x00000000
Sema:     .word 0x00000001

   ldr    r3, =Sema
   ldr    r4, =Count
   mov    r1, #1
for_enter:
   cmp    r1, #100
   bgt    end_for_enter
wait_1:
   ldrex  r0, [r3]
   cbz    r0, wait_1 ; if Semaphore = 0
   sub    r0, #1     ; dec Semaphore
   strex  r0, [r3]   ; try update
   cbnz   r0, wait_1 ; if touched
   dmb               ; sync memory
   …
signal_1:
   ldrex  r0, [r3]
   add    r0, #1         ; inc Semaphore
   strex  r0, [r3]       ; try update
   cbnz   r0, signal_1   ; if touched
   dmb                   ; sync memory
   add    r1, #1
   b      for_enter
end_for_enter:
```

Any context switch needs to clear reservations

Critical section

```
   ldr    r3, =Sema
   ldr    r4, =Count
   mov    r1, #1
for_leave:
   cmp    r1, #100
   bgt    end_for_leave
wait_2:
   ldrex  r0, [r3]
   cbz    r0, wait_2 ; if Semaphore = 0
   sub    r0, #1     ; dec Semaphore
   strex  r0, [r3]   ; try update
   cbnz   r0, wait_2 ; if touched
   dmb               ; sync memory
   …
signal_2:
   ldrex  r0, [r3]
   add    r0, #1         ; inc Semaphore
   strex  r0, [r3]       ; try update
   cbnz   r0, signal_2   ; if touched
   dmb                   ; sync memory
   add    r1, #1
   b      for_leave
end_for_leave:
```

Critical section

Semaphores

```
S : Semaphore := 1;
```

```
task body Pi is
begin
 loop
  ------ non_critical_section_i;
  wait (S);
   ------ critical_section_i;
  signal (S);
 end loop;
end Pi;
```

```
task body Pj is
begin
 loop
  ------ non_critical_section_j;
  wait (S);
   ------ critical_section_j;
  signal (S);
 end loop;
end Pi;
```

☞ Works?

Semaphores

```
S : Semaphore := 1;
```

```
task body Pi is
begin
 loop
  ------ non_critical_section_i;
  wait (S);
   ------ critical_section_i;
  signal (S);
 end loop;
end Pi;
```

```
task body Pj is
begin
 loop
  ------ non_critical_section_j;
  wait (S);
   ------ critical_section_j;
  signal (S);
 end loop;
end Pi;
```

- ☞ Mutual exclusion!, No deadlock!, No global live-lock!
- ☞ Works for any dynamic number of processes
- ☞ Individual starvation possible!

Semaphores

```
S1, S2 : Semaphore := 1;
```

```
task body Pi is
begin
 loop
  ------ non_critical_section_i;
  wait (S1);
  wait (S2);
   ------ critical_section_i;
  signal (S2);
  signal (S1);
 end loop;
end Pi;
```

```
task body Pj is
begin
 loop
  ------ non_critical_section_j;
  wait (S2);
  wait (S1);
   ------ critical_section_j;
  signal (S1);
  signal (S2);
 end loop;
end Pi;
```

☞ Works too?

Semaphores

```
S1, S2 : Semaphore := 1;
```

```
task body Pi is
begin
 loop
  ------ non_critical_section_i;
  wait (S1);
  wait (S2);
   ------ critical_section_i;
  signal (S2);
  signal (S1);
 end loop;
end Pi;
```

```
task body Pj is
begin
 loop
  ------ non_critical_section_j;
  wait (S2);
  wait (S1);
   ------ critical_section_j;
  signal (S1);
  signal (S2);
 end loop;
end Pi;
```

- ☞ Mutual exclusion!, No global live-lock!
- ☞ Works for any dynamic number of processes.
- ☞ Individual starvation possible!
- ☞ Deadlock possible!

Semaphores

```
S1, S2 : Semaphore := 1;
```

```
task body Pi is
begin
 loop
  ------ non_critical_section_i;
  wait (S1);
  wait (S2);
   ------ critical_section_i;
  signal (S2);
  signal (S1);
 end loop;
end Pi;
```

```
task body Pj is
begin
 loop
  ------ non_critical_section_j;
  wait (S2);
  wait (S1);
   ------ critical_section_j;
  signal (S1);
  signal (S2);
 end loop;
end Pi;
```

- ☞ Mutual exclusion!, No global live-lock!
- ☞ Works for any dynamic number of processes.
- ☞ Individual starvation possible!
- ☞ Deadlock possible!

Concurrent programming languages offer **higher abstraction** and **safer** synchronization mechanisms.

Summary

Aynchronism

- **Interrupts & Exceptions**
  - Concept
  - Hardware/Software interaction
  - Recursive interrupts
- **Concurrency & Synchronization**
  - Race conditions
  - Synchronization
  - Passing data